GROT Yanis GROUPE 3

TPR203

Table des matières :

- Introduction
- TP1 : Le service DHCP
- TP2 : Le service SSH
- TP3 : Configuration d’un serveur
- TP4 : Configuration d’un serveur FTP répliqué
- Conclusion

Introduction :

Ces travaux pratiques s'inscrivent dans le cadre du module R203 Bases des services réseaux, à l'IUT de Villetaneuse au sein du département Réseaux et Télécommunications. Ils nous permettent de mettre en pratique les connaissances théoriques acquises en cours en nous faire la configuration de services réseaux fondamentaux, dans des environnements que l'on peut retrouver en entreprise. L'outil principal que nous allons utiliser tout au long de ces séances est Marionnet, un émulateur réseau qui nous permet de construire et de faire fonctionner des topologies complètes avec des hôtes, des routeurs, des switchs et des passerelles, le tout tournant sur des machines virtuelles Linux. Les services y sont gérés via des scripts situés dans le répertoire /etc/init.d/, ce qui nous permet de nous familiariser avec la gestion de services dans un environnement Linux assez classique.

Le module est composé de quatre travaux pratiques, chacun centré sur un service réseau différent. Le premier TP porte sur le service DHCP, dont le rôle est d'attribuer automatiquement des adresses IP aux machines d'un réseau. Nous allons y configurer un serveur DHCP, mettre en place un relai pour transmettre les requêtes entre différents sous-réseaux, attribuer des adresses fixes à certaines machines grâce à leur adresse MAC, et même indiquer aux clients l'adresse d'un serveur NTP pour synchroniser leurs horloges. Le deuxième TP est consacré au service SSH. Nous commençons par comparer SSH à Telnet pour bien comprendre pourquoi Telnet n'est plus utilisé, notamment parce qu'il transmet les mots de passe en clair sur le réseau. Nous explorons ensuite des fonctionnalités très utiles de SSH comme l'authentification par clé, la copie de fichiers à distance avec scp, l'automatisation de commandes via des scripts, et la création de tunnels SSH pour sécuriser un trafic qui ne l'est pas de base. Le troisième TP nous fait travailler sur Apache, un serveur web très utilisé. Nous y configurons un serveur capable d'héberger plusieurs sites en même temps grâce aux hôtes virtuels, nous créons des pages d'erreur personnalisées, nous activons des journaux pour suivre les accès et les erreurs, nous écrivons des scripts CGI pour générer du contenu dynamique, et nous protégeons certaines parties du site avec un système d'authentification. Enfin, le quatrième TP porte sur la mise en place d'un serveur FTP répliqué. Nous utilisons le service proftpd pour le serveur FTP, Apache pour créer un proxy qui sert d'intermédiaire entre le client et le serveur FTP, et l'outil rsync pour synchroniser régulièrement un serveur miroir avec le serveur principal, ce qui permet de maintenir le service même en cas de panne.

GROT Yanis GROUPE 3

TP1 : Le service DHCP

Table des matières :

- Introduction
- Exercice 1 — Création du réseau
- Exercice 2 — Configuration statique des interfaces
- Exercice 3 — Configuration du réseau des utilisateurs
- Exercice 4 — Configuration du réseau des serveurs
- Exercice 5 — Configuration du serveur NTP
- Exercice 6 — Configuration du client
- Conclusion

Introduction :

Dans ce premier TP du module R203 « Bases des services réseaux », nous avons étudié la mise en place d'un service DHCP (Dynamic Host Configuration Protocol). Ce protocole permet d'attribuer automatiquement des adresses IP et d'autres paramètres réseau aux machines connectées à un réseau. Pour réaliser ce TP, nous avons utilisé l'outil Marionnet, qui permet de simuler un réseau informatique composé de plusieurs machines virtuelles, telles que des serveurs, des clients et un routeur. Nous avons également utilisé des commandes Linux pour configurer les interfaces réseau et gérer les services.

L'objectif principal de ce TP est de comprendre le fonctionnement d'un serveur DHCP et de savoir le configurer dans un réseau comportant plusieurs sous-réseaux. Il s'agit également de mettre en place un relai DHCP afin de permettre la communication entre les différents réseaux, ainsi que de configurer des options supplémentaires comme l'attribution d'adresses IP fixes ou la synchronisation avec un serveur NTP.

Pour atteindre ces objectifs, plusieurs étapes ont été réalisées. Tout d'abord, nous avons créé l'architecture réseau demandée dans Marionnet et configuré les adresses IP statiques des équipements nécessaires. Ensuite, nous avons configuré le serveur DHCP afin qu'il attribue automatiquement des adresses IP aux machines clientes. Nous avons ensuite mis en place un relai DHCP pour permettre l'attribution d'adresses sur un second réseau. Enfin, nous avons testé le bon fonctionnement du service DHCP ainsi que certaines options avancées, comme l'attribution d'une adresse IP fixe et la configuration d'un serveur NTP.

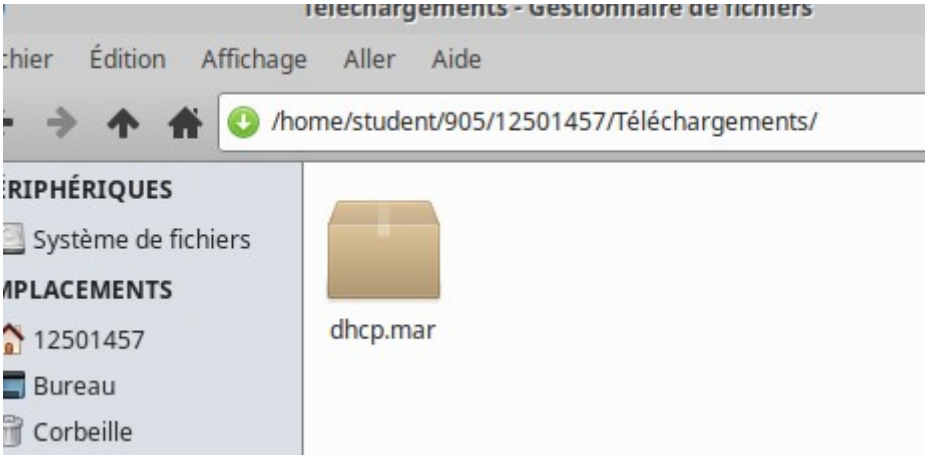
Exercice 1 — Création du réseau

Objectif : L’objectif de cet exercice est de mettre en place l’architecture réseau demandée à l’aide du logiciel Marionnet. Cette étape permet de créer un environnement de travail simulant un réseau réel composé de plusieurs sous-réseaux interconnectés par un routeur. Elle constitue une base indispensable pour la suite du TP, car le bon fonctionnement du service DHCP dépend directement de la structure du réseau et du câblage entre les équipements. Cet exercice vise également à se familiariser avec l’ajout d’équipements réseau virtuels et leur démarrage dans un environnement simulé.

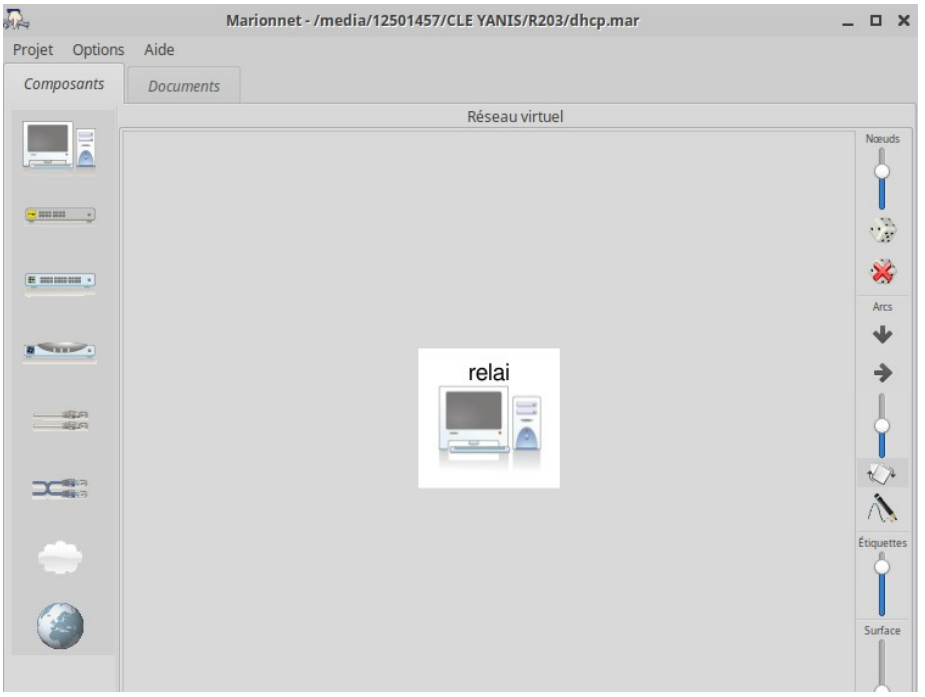
Étapes réalisées

- **Téléchargement du projet** : Récupération du fichier de base à l'URL indiquée.
- **Ouverture du projet dans Marionnet** : Chargement du fichier marionnet_dhcp.mar.
- **Ajout des équipements réseau** : Intégration du routeur (avec terminal Unix), de la passerelle G (réseau 10.7.3.0/24) et respect du câblage des ports.
- **Démarrage et ouverture de session** : Lancement de tous les équipements et connexion en tant que root sur chaque hôte.

1.1 Téléchargez le projet à l’URL indiquée.



Le projet a bien été téléchargé.
I 1.2 Lancez marionnet et ouvrez le projet téléchargé.



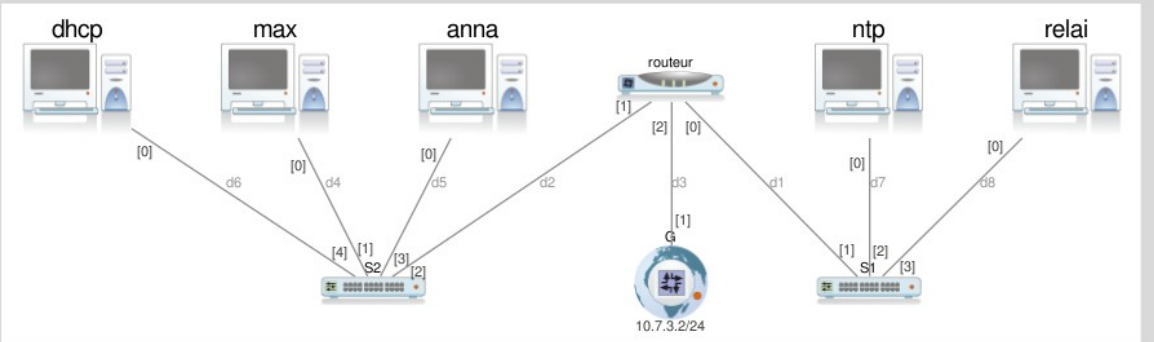
Le fichier marionnet dhcp.mar a été ouvert correctement et sans problème.

I 1.3 Ajoutez les équipements pour obtenir la réseau de la figure en introduction. Attention :

— À la création du routeur, cochez la case Montrer le terminal Unix et laissez l'IP proposée par défaut.

— À la création de la passerelle G, choisissez l'adresse de réseau 10.7.3.0/24. Marionnet attribuera l'IP 10.7.3.2 à la passerelle (voir figure).

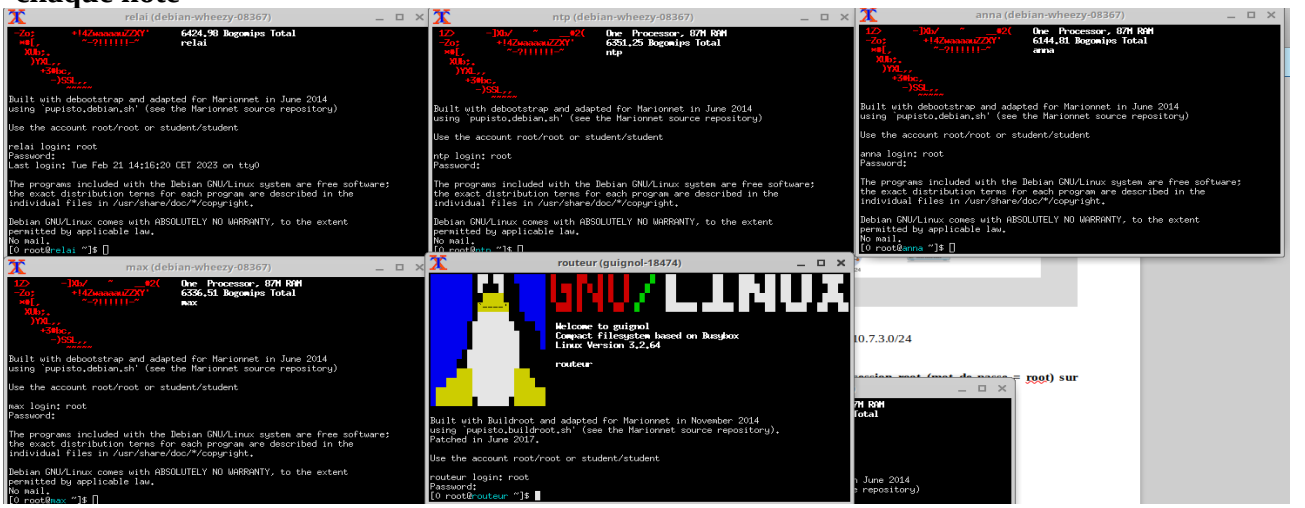
— Enfin, respectez le câblage de la figure pour le routeur : son port 0 mène au switch des serveurs, son port 1mène au switch des utilisateurs et son port 2 mène à la passerelle.



Le schéma a été réalisé et j'ai choisi l'adresse de réseau : 10.7.3.0/24

L’adresse de la passerelle : 10.7.3.2

I 1.4 Démarrez tous les équipements et ouvrez une session root (mot de passe = root) sur chaque hôte



Exercice 2 — Configuration statique des interfaces

Objectif : L’objectif de cet exercice est de configurer manuellement les interfaces réseau des équipements essentiels, notamment le routeur, le relai DHCP et le serveur DHCP. Contrairement aux machines clientes, ces équipements doivent posséder des adresses IP fixes afin d’assurer une communication stable dans le réseau. Cet exercice permet également d’ajouter des routes par défaut afin d’assurer la communication entre les différents sous-réseaux et de vérifier la connectivité entre les machines grâce à des tests réseau. Il s’agit d’une étape fondamentale pour garantir le bon fonctionnement du service DHCP dans la suite du TP.

Étapes réalisées :

- Attribution des adresses IP statiques : Configuration manuelle des adresses IP pour le routeur (sur 3 interfaces), le relai et le serveur DHCP.

- Configuration du routage : Ajout d'une route par défaut sur le routeur, le relai et le serveur DHCP pour permettre la communication entre sous-réseaux.

- Vérification de la connectivité : Test de communication entre le serveur DHCP et le relai via des pings.

- Test de résolution DNS : Vérification de l'accès extérieur (DNS de l'université) depuis la machine relai

Commandes utilisées : ip addr, ip link set, ip route add, ping, host

I 2.1 Attribuez au routeur, au relai et au serveur dhcp les IP indiquées dans le tableau. Utilisez les commandes suivantes en remplaçant @ip et ethX par l’IP et l’interface voulues :

Pour le routeur :
- eth0: 10.7.1.254
- eth1 : 10.7.2.254
- eth2 : 10.7.3.254

```
[0 root@routeur ~]$ ip addr add 10.7.1.254/24 dev eth0
[0 root@routeur ~]$ ip link set eth0 up
[0 root@routeur ~]$ ip addr add 10.7.2.254/24 dev eth1
[0 root@routeur ~]$ ip link set eth1 up
[0 root@routeur ~]$ ip addr add 10.7.3.254/24 dev eth2
[0 root@routeur ~]$ ip link set eth2 up
[0 root@routeur ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN group defaul
t
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: bond0: <BROADCAST,MULTICAST,MASTER> mtu 1500 qdisc noop state DOWN group defa
ult
    link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff
3: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group default
    link/ether 26:e1:29:b3:1d:80 brd ff:ff:ff:ff:ff:ff
4: tunl0: <NOARP> mtu 1480 qdisc noop state DOWN group default
    link/ipip 0.0.0.0 brd 0.0.0.0
5: sit0: <NOARP> mtu 1480 qdisc noop state DOWN group default
    link/sit 0.0.0.0 brd 0.0.0.0
6: ip6tnl0: <NOARP> mtu 1452 qdisc noop state DOWN group default
    link/tunnel6 :: brd ::
7: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNO
WN group default qlen 1000
    link/ether 02:04:06:9b:79:95 brd ff:ff:ff:ff:ff:ff
    inet 10.7.1.254/24 scope global eth0
8: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNO
WN group default qlen 1000
    link/ether 02:04:06:af:67:14 brd ff:ff:ff:ff:ff:ff
    inet 10.7.2.254/24 scope global eth1
9: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNO
WN group default qlen 1000
    link/ether 02:04:06:54:3a:10 brd ff:ff:ff:ff:ff:ff
    inet 10.7.3.254/24 scope global eth2
```

Pour le relai :
- eth0: 10.7.1.100

Pour le dhcp :
- eth 0 : 10.7.2.100

Pour le relai :

```
[0 root@relai ~]$ ip addr add 10.7.1.100/24 dev eth0
[0 root@relai ~]$ ip link set eth0 up
[0 root@relai ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: bond0: <BROADCAST,MULTICAST,MASTER> mtu 1500 qdisc noop state DOWN
    link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff
3: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether ba:64:92:e4:ba:0b brd ff:ff:ff:ff:ff:ff
4: tunl0: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: sit0: <NOARP> mtu 1480 qdisc noop state DOWN
    link/sit 0.0.0.0 brd 0.0.0.0
6: ip6tnl0: <NOARP> mtu 1452 qdisc noop state DOWN
    link/tunnel6 :: brd ::
7: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNO
WN qlen 1000
    link/ether 02:04:06:97:7a:e7 brd ff:ff:ff:ff:ff:ff
    inet 10.7.1.100/24 scope global eth0
    inet6 fe80::4:6ff:fe97:7ae7/64 scope link
       valid_lft forever preferred_lft forever
```

Pour le dhcp :

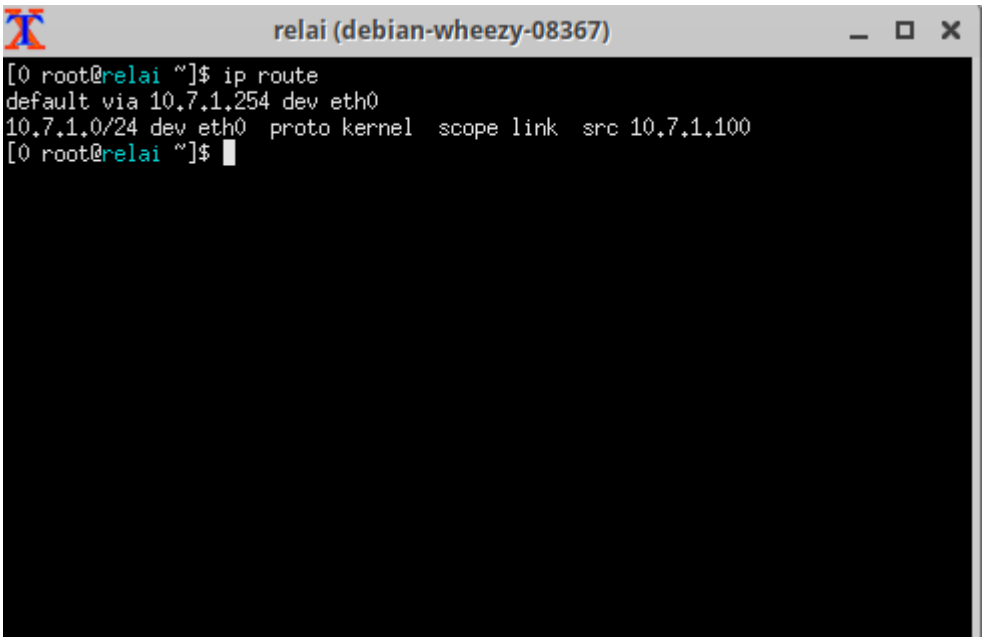
```
[0 root@dhcp ~]$ ip addr add 10.7.2.100/24 dev eth0
[0 root@dhcp ~]$ ip link set eth0 up
[0 root@dhcp ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: bond0: <BROADCAST,MULTICAST,MASTER> mtu 1500 qdisc noop state DOWN
    link/ether 00:00:00:00:00:00 brd ff:ff:ff:ff:ff:ff
3: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 06:08:06:bc:e1:21 brd ff:ff:ff:ff:ff:ff
4: tunl0: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: sit0: <NOARP> mtu 1480 qdisc noop state DOWN
    link/sit 0.0.0.0 brd 0.0.0.0
6: ip6tnl0: <NOARP> mtu 1452 qdisc noop state DOWN
    link/tunnel6 :: brd ::
7: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNO
WN qlen 1000
    link/ether 02:04:06:5d:ca:9e brd ff:ff:ff:ff:ff:ff
    inet 10.7.2.100/24 scope global eth0
    inet6 fe80::4:6ff:fe5d:ca9e/64 scope link tentative
       valid_lft forever preferred_lft forever
[0 root@dhcp ~]$
```

2.2 Attribuez une route par défaut au routeur, au relai et au serveur dhcp. Utilisez la commande suivante :
- ip route add default via @ip

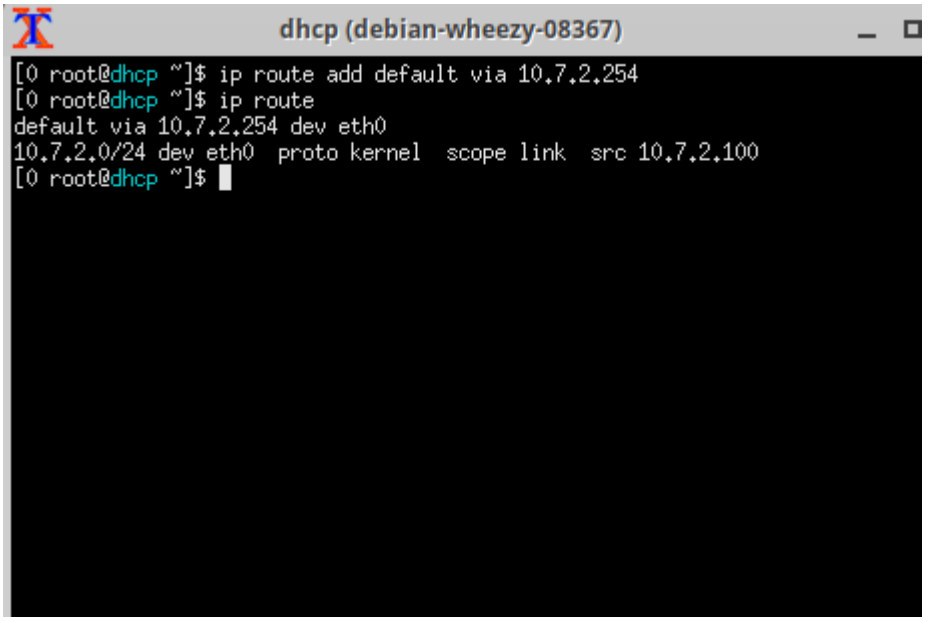
Sur le routeur :

```
[0 root@routeur ~]$ ip route add default via 10.7.3.2
[0 root@routeur ~]$ ip route
default via 10.7.3.2 dev eth2
10.7.1.0/24 dev eth0  proto kernel  scope link  src 10.7.1.254
10.7.2.0/24 dev eth1  proto kernel  scope link  src 10.7.2.254
10.7.3.0/24 dev eth2  proto kernel  scope link  src 10.7.3.254
[0 root@routeur ~]$
```

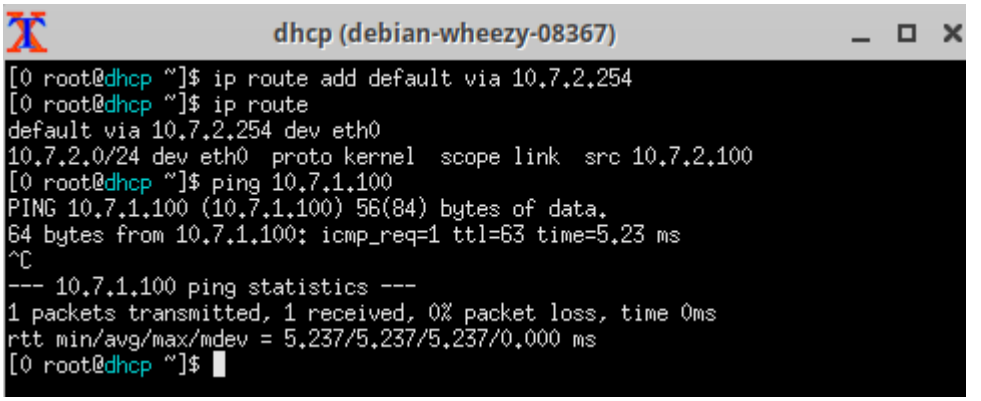
Sur relai :



Sur DHCP :

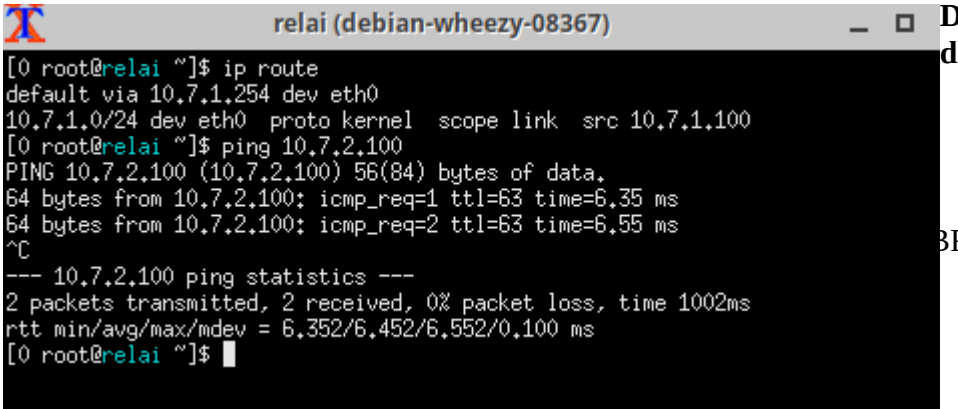


I 2.3 Vérifiez que le relai et le serveur dhcp peuvent bien s'échanger des messages ping.



De dhcp à relai :

De relai à dhcp :



Le relai et le dhcp peuvent bien communiquer entre eux.

2.4 Sur la machine relai, utilisez la commande ci-dessous qui interroge le serveur DNS 193.54.26.1 (un des serveurs DNS de l'université) pour obtenir l'IP associée au serveur web de l'université.



Relai a accès au site de l'université.

Exercice 3 — Configuration du réseau des utilisateurs

Objectif : L’objectif de cet exercice est de mettre en place un serveur DHCP afin d’attribuer automatiquement des adresses IP aux machines clientes du réseau des utilisateurs. Contrairement à la configuration statique réalisée précédemment, cette étape permet d’automatiser la gestion des adresses IP, ce qui simplifie considérablement l’administration du réseau. Cet exercice vise également à comprendre le fonctionnement du protocole DHCP, notamment le processus d’attribution des baux, ainsi que la configuration des paramètres essentiels tels que la plage d’adresses IP, la passerelle par défaut et les serveurs DNS. Enfin, il s’agit de vérifier que les clients reçoivent correctement leur configuration réseau et peuvent communiquer avec les autres machines et accéder à Internet.

Étapes réalisées :
- Définition de l'interface d'écoute : Modification du fichier /etc/default/isc-dhcp-server pour spécifier eth0

- **Initialisation du fichier de configuration** : Suppression et recréation du fichier de configuration DHCP.

- Paramétrage du serveur DHCP : Édition du fichier pour définir les serveurs DNS, la plage d'IP, la passerelle et la durée du bail.

- Démarrage du service : Redémarrage du démon `dhcpd` pour appliquer les changements.

- Demande de bail client : Obtention automatique d'adresses IP sur les hôtes Anna et Max.

- **Vérification finale** : Tests de ping entre clients et téléchargement d'une page web pour valider l'accès internet.

Commandes utilisées : nano, rm, restart, dhclient, ip route, cat, wget

I 3.1 Éditez le fichier /etc/default/isc-dhcp-server avec l’éditeur nano

I 3.2 Recherchez la ligne INTERFACES="" et insérez eth0 entre les guillemets.

I 3.3 Sauvegardez le fichier (ctrl+o puis entrée) puis quittez nano (ctrl+x)

```
# Defaults for isc-dhcp-server initscript
# sourced by /etc/init.d/isc-dhcp-server
# installed at /etc/default/isc-dhcp-server by the maintainer scripts

#
# This is a POSIX shell fragment
#

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPD_CONF=/etc/dhcp/dhcpd.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPD_PID=/var/run/dhcpd.pid

# Additional options to start dhcpd with.
#       Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
#       Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACES="eth0"
```

I 3.4 Supprimez le fichier de configuration dhcp puis recréez le

```
[0 root@dhcp ~]$ rm /etc/dhcp/dhcpd.conf
[0 root@dhcp ~]$ nano /etc/dhcp/dhcpd.conf
Use "fg" to return to nano
```

I 3.5 Avec nano, éditez le fichier pour qu'il ait le contenu suivant :

```
option domain-name-servers 193.54.26.1;
subnet 10.7.2.0 netmask 255.255.255.0 {
        range 10.7.2.1 10.7.2.10;
        option routers 10.7.2.254;
        max-lease-time 3600;
}
```

3.6 Redémarrez le service dhcpd.

```
[0 root@dhcp ~]$ /etc/init.d/dhcpd restart
[FAIL] Stopping ISC DHCP server: dhcpd failed!
[ ok ] Starting ISC DHCP server: dhcpd.
[0 root@dhcp ~]$
```

I 3.7 Sur les hôtes anna et max, demandez un bail pour l'interface eth0 :

Sur anna :

```
[0 root@anna ~]$ dhclient eth0
Internet Systems Consortium DHCP Client 4.2.2
Copyright 2004-2011 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/02:04:06:cd:e3:a4
Sending on   LPF/eth0/02:04:06:cd:e3:a4
Sending on   Socket/fallback
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 4
DHCPREQUEST on eth0 to 255.255.255.255 port 67
DHCPOFFER from 10.7.2.100
DHCPACK from 10.7.2.100
bound to 10.7.2.1 -- renewal in 1473 seconds.
[0 root@anna ~]$
```

Sur max :

```
[0 root@max ~]$ dhclient eth0
Internet Systems Consortium DHCP Client 4.2.2
Copyright 2004-2011 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/02:04:06:a3:ca:84
Sending on   LPF/eth0/02:04:06:a3:ca:84
Sending on   Socket/fallback
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 3
DHCPREQUEST on eth0 to 255.255.255.255 port 67
DHCPOFFER from 10.7.2.100
DHCPACK from 10.7.2.100
bound to 10.7.2.2 -- renewal in 1798 seconds.
[0 root@max ~]$
```

I 3.8 Anna peut envoyer un message ping à max et au relai :

anna vers max :

```
[0 root@anna ~]$ ping 10.7.2.2
PING 10.7.2.2 (10.7.2.2) 56(84) bytes of data.
64 bytes from 10.7.2.2: icmp_req=1 ttl=64 time=4.54 ms
^C
--- 10.7.2.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.542/4.542/4.542/0.000 ms
[0 root@anna ~]$
```

anna vers relai :

```
[0 root@anna ~]$ ping 10.7.1.100
PING 10.7.1.100 (10.7.1.100) 56(84) bytes of data.
64 bytes from 10.7.1.100: icmp_req=1 ttl=63 time=7.75 ms
^C
--- 10.7.1.100 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 7.757/7.757/7.757/0.000 ms
[0 root@anna ~]$
```

I 3.9 Anna peut télécharger la page d'accueil de google avec wget (client web en mode terminal) :

```
[0 root@anna ~]$ wget http://www.google.fr
--2026-03-30 10:20:37--  http://www.google.fr/
Resolving www.google.fr (www.google.fr)... 172.217.22.35, 2a00:1450:4007:816::20
03
Connecting to www.google.fr (www.google.fr)|172.217.22.35|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: http://www.google.com/ [following]
--2026-03-30 10:20:37--  http://www.google.com/
Resolving www.google.com (www.google.com)... 142.251.152.119, 142.251.153.119, 1
42.251.154.119, ...
Connecting to www.google.com (www.google.com)|142.251.152.119|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
Saving to: `index.html'

    [ <=>                                   ] 78,753      --.-K/s   in 0.01s

2026-03-30 10:20:37 (5.92 MB/s) - `index.html' saved [78753]

[0 root@anna ~]$
```

Le fichier index.html a bien été téléchargé.

Q 3.1 Sur anna, quelle commande peut-on exécuter pour vérifier que la ligne 4 du fichier de configuration du service a bien été prise en compte ? Testez la commande

la commande a utilisé est : ip route



On voit que la route par défaut pour anna est 10.7.2.254, on peut donc constater que la ligne numéro 4 dans le fichier de configuration dhcp a bien été prise en compte car dans le fichier dhcp nous avions mis option routers 10.7.2.254. La route par défaut lui a été fourni automatiquement avec le bail.

Q 3.2 Sur anna, quel fichier peut-on consulter pour vérifier que la ligne 1 du fichier de configuration du service a bien été prise en compte ? Consultez ce fichier.

La commande a utiliser est : cat /*etc*/resolv.conf



Ici l'adresse DNS a bien été communiqué par le serive dhcp grâce à le demande de bail avec dhclient eth0. La ligne 1 était option domain-name-servers 193.54.26.1; qui donnait comme adresse 193.54.26.1. Il s'agit de la même adresse donc aucun problème.

Q 3.3 Expliquez pourquoi ces deux lignes (1 et 4) sont nécessaires pour que le test de l'instruction I 3.9 fonctionne :

la ligne 1 est nécessaire pour qu'anna puisse résoudre www.google.fr comme adresse IP : 193.54.26.1. La ligne 4 quant à elle est nécessaire piur qu'anna puisse savoir où envoyer ces paquets pour atteindre l'internet.

Exercice 4 — Configuration du réseau des serveurs

Objectif : L’objectif de cet exercice est de permettre l’attribution d’adresses IP à des machines situées dans un autre sous-réseau que celui du serveur DHCP. Pour cela, il est nécessaire de mettre en place un relai DHCP, qui a pour rôle de transmettre les requêtes DHCP des clients vers le serveur situé sur un réseau différent. Cet exercice permet ainsi de comprendre le fonctionnement du relai DHCP dans un environnement multi-réseaux, ainsi que les mécanismes de communication entre les différentes entités du réseau. Il vise également à vérifier que les machines du second réseau peuvent obtenir automatiquement une configuration réseau malgré la séparation des sous-réseaux.

Étapes réalisées :

- Configuration du relai DHCP : Paramétrage du fichier /etc/default/isc-dhcp-relay avec l'IP du serveur DHCP et activation sur l'interface adéquate

- **Ajout d'un sous-réseau au serveur** : Déclaration du nouveau réseau (10.7.1.0) dans la configuration du serveur DHCP.

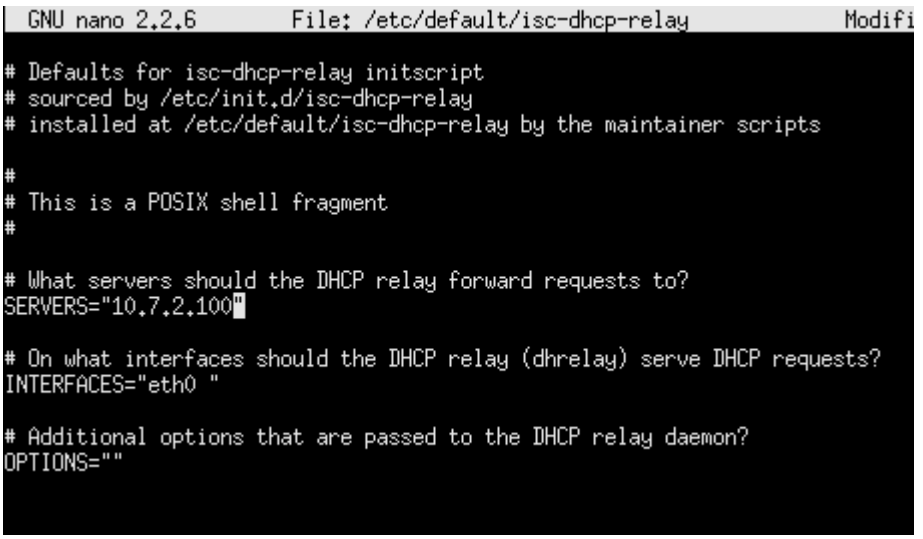
- **Test d'attribution via relai** : Demande de bail sur le serveur NTP situé sur le second réseau.

- **Analyse de trafic** : Capture des trames (Wireshark) pour observer le processus DORA et identifier l'adresse de l'agent relai.

- **Attribution d'une IP fixe** : Configuration du serveur DHCP pour assigner une IP spécifique basée sur l'adresse MAC du serveur NTP.

Commandes utilisées : nano, restart, dhclient -r, wireshark, ip link show

4.1 Répétez les instructions I 3.1 à I 3.3 mais sur le fichier /etc/default/isc-dhcp-relay. Modifiez ausi la ligne SERVERS="" en insérant l’IP du serveur dhcp entre les guillemets.



I 4.2 Redémarrez le service isc-dhcp-relay

```
[0 root@relai ~]$ /etc/init.d/isc-dhcp-relay restart
[0 root@relai ~]$
```

Le service a été redémarré, rien ne s'affiche ça fonctionne correctement.

I 4.3 Dans le fichier de configuration du service dhcp, copiez-collez les lignes déclarant le réseau 10.N.2.0 et adaptez les pour le réseau 10.N.1.0 avec les mêmes contraintes (10 machines max. et même durée max. de bail).

```
option domain-name-servers 193.54.26.1;
subnet 10.7.2.0 netmask 255.255.255.0 {
        range 10.7.2.1 10.7.2.10;
        option routers 10.7.2.254;
        max-lease-time 3600;
}

subnet 10.7.1.0 netmask 255.255.255.0 {
        range 10.7.1.1 10.7.1.10;
        option routers 10.7.1.254;
        max-lease-time 3600;
}
```

Voici la configuration, nous avons rajouté un autre subnet pour les machines appartenant au réseau 10.7.1.0. Ces machines passeront par la route par défaut : 10.7.1.254.

I 4.4 Redémarrez le service dhcpd

```
[0 root@dhcp ~]$ nano /etc/dhcp/dhcpd.conf
[0 root@dhcp ~]$ /etc/init.d/dhcpd restart
[FAIL] Stopping ISC DHCP server: dhcpd failed!
[ ok ] Starting ISC DHCP server: dhcpd.
[0 root@dhcp ~]$
```

I 4.5 Demandez un bail sur le serveur ntp pour vérifier qu'il obtient bien une IP

```
[0 root@ntp ~]$ dhclient eth0
Internet Systems Consortium DHCP Client 4.2.2
Copyright 2004-2011 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/02:04:06:8e:8a:13
Sending on   LPF/eth0/02:04:06:8e:8a:13
Sending on   Socket/fallback
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 8
DHCPREQUEST on eth0 to 255.255.255.255 port 67
DHCPOFFER from 10.7.1.100
DHCPACK from 10.7.1.100
bound to 10.7.1.1 -- renewal in 1704 seconds.
[0 root@ntp ~]$
```

Le serveur obtient bien une adresse IP : 10.7.1.1.

Q 4.1 De quel hôte le serveur ntp a-t-il reçu les messages OFFER et ACK ?

Le serveur ntp hôte a reçu ces messages du serveur dhcp, le relai ne génère pas ces messages ils les transmet.

I 4.6 Sur le serveur ntp, résiliez le bail (dhclient -r eth0).

```
[0 root@ntp ~]$ dhclient -r eth0
Internet Systems Consortium DHCP Client 4.2.2
Copyright 2004-2011 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/02:04:06:8e:8a:13
Sending on   LPF/eth0/02:04:06:8e:8a:13
Sending on   Socket/fallback
DHCPRELEASE on eth0 to 10.7.2.100 port 67
[0 root@ntp ~]$
```

I 4.7 Tout en capturant les trames sur le serveur dhcp (avec wireshark), redemandez un bail sur le serveur ntp.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.7.1.100	10.7.2.100	DHCP	342	DHCP Discover - Transaction ID 0xf8682f7c
2	0.015080000	10.7.2.100	10.7.1.1	ICMP	62	Echo (ping) request id=0xd86a, seq=0/0, ttl=64
3	1.022360000	10.7.2.100	10.7.1.100	DHCP	342	DHCP Offer - Transaction ID 0xf8682f7c
4	1.030616000	10.7.1.100	10.7.2.100	DHCP	342	DHCP Request - Transaction ID 0xf8682f7c
5	1.043583000	10.7.2.100	10.7.1.100	DHCP	342	DHCP ACK - Transaction ID 0xf8682f7c
6	5.013312000	BbnInter_af:67:14	BbnInter_5d:ca:9e	ARP	42	Who has 10.7.2.100? Tell 10.7.2.254
7	5.013331000	BbnInter_5d:ca:9e	BbnInter_af:67:14	ARP	42	10.7.2.100 is at 02:04:06:5d:ca:9e

D'abord, on observe un échange DHCP complet (Discover, Offer, Request, ACK), qui permet à un appareil d'obtenir automatiquement une configuration réseau auprès d'un serveur. Ensuite, on voit une requête ARP, où un appareil demande l'adresse physique (MAC) associée à l'adresse IP 10.7.2.100 afin de pouvoir communiquer directement avec lui sur le réseau local. En résumé, c'est le cycle typique d'un appareil qui se connecte au réseau et cherche à identifier ses voisins.

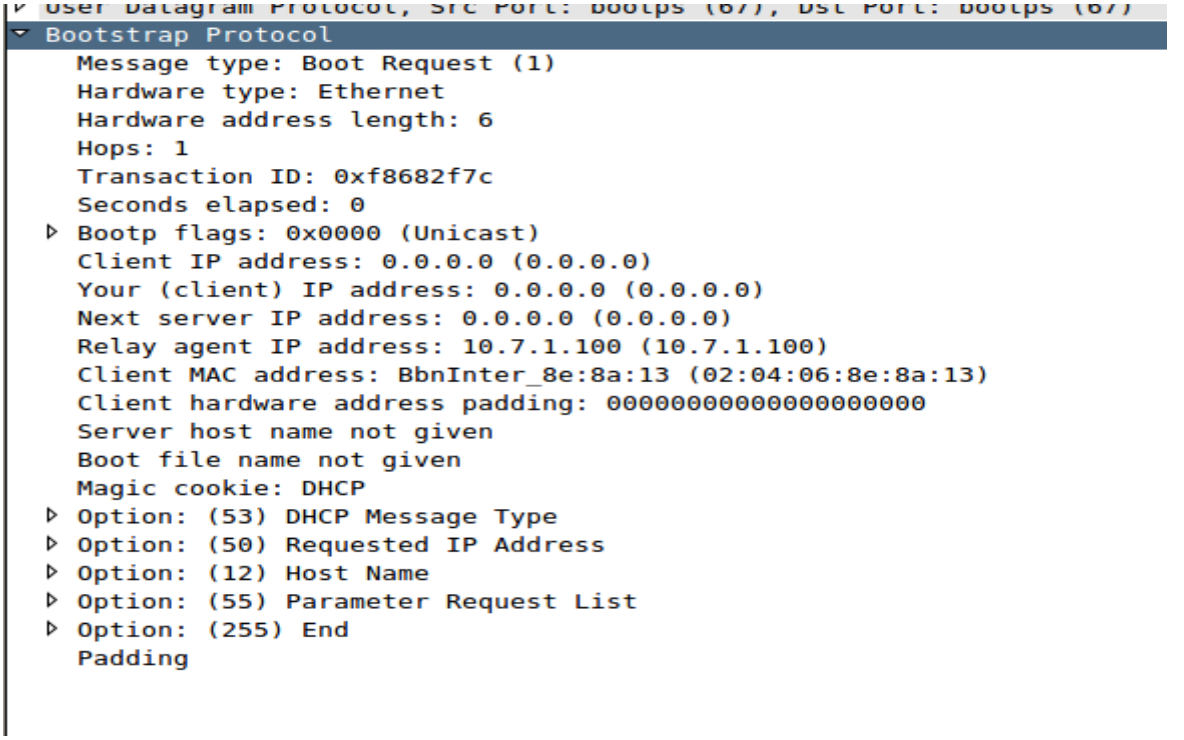
4.8 Sur le serveur dhcp, arrêtez la capture puis ajoutez le filtre bootp pour ne voir que les messages dhcp.

Filter: bootp Expression... Clear Apply Save

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.7.1.100	10.7.2.100	DHCP	342	DHCP Discover - Transaction ID 0xf8682f7c
3	1.022360000	10.7.2.100	10.7.1.100	DHCP	342	DHCP Offer - Transaction ID 0xf8682f7c
4	1.030616000	10.7.1.100	10.7.2.100	DHCP	342	DHCP Request - Transaction ID 0xf8682f7c
5	1.043583000	10.7.2.100	10.7.1.100	DHCP	342	DHCP ACK - Transaction ID 0xf8682f7c

On y voit les quatre étapes clés : le client commence par chercher un serveur (Discover), le serveur propose une configuration (Offer), le client confirme son choix (Request) et le serveur valide l'opération (ACK). Ce processus s'appelle DORA.

Q 4.2 En analysant le contenu du message DISCOVER reçu par le serveur dhcp (affichez pour cela le détail du message Bootstrap Protocol dans le pannneau du milieu de wireshark), trouvez l'information qui lui permet de déterminer que la requête vient d'un hôte du réseau 10.N.1.0 et pas du réseau 10.N.2.0 (et qu'il doit donc lui attribuer une IP dans le réseau 10.N.1.0). Ne regardez pas dans les options pour trouver cette information.



L'information qui permet de déterminer qu'il s'agit du réseau 10.7.1.0 et pas 10.7.2.0 est :
relay agent IP address : 10.7.1.100. Cette adresse correspond à l'adresse IP du relai. Lorsqu'un client envoie un message discover en diffusion sur un réseau différent de celui du serveur alors le relai dhcp intercepte la requête et inscrit donc sa propre adresse IP dans le champ relay agent ip address avant de le transmettre au serveur.

Q 4.3 Regardez maintenant les options du DISCOVER. Analysez les pour trouver la liste des informations de configuration demandées au serveur. Citez 5 informations que vous connaissez et donnez les codes correspondant

Dans le message DHCP Discover, la liste des informations de configuration demandées se trouve sous l'option (55) Parameter Request List. Le client utilise cette option pour signaler au serveur les paramètres réseau dont il a besoin pour fonctionner correctement.

Voici les 5 informations :

- Subnet Mask : Le masque de sous-réseau qui est indispensable pour définir les limites du réseau local.

- Router : L'adresse de la passerelle par défaut (Default Gateway) pour sortir du réseau local.

- Domain Name Server : Les adresses des serveurs DNS pour la résolution de noms de domaine.

- Domain Name : Le nom du domaine local auquel l'hôte appartient.

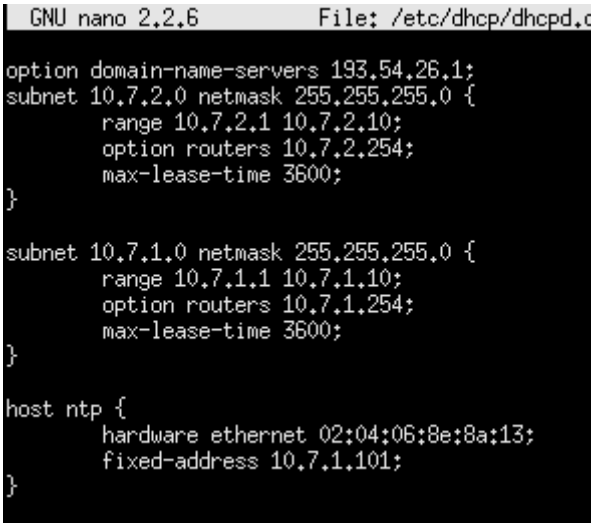
- Network Time Protocol Servers : Les serveurs NTP permettant de synchroniser l'horloge de l'appareil.

4.9 Sur le serveur ntp, trouvez l’adresse MAC de l’interface eth0



L’adresse MAC est : 02:04:06:8e:8a:13

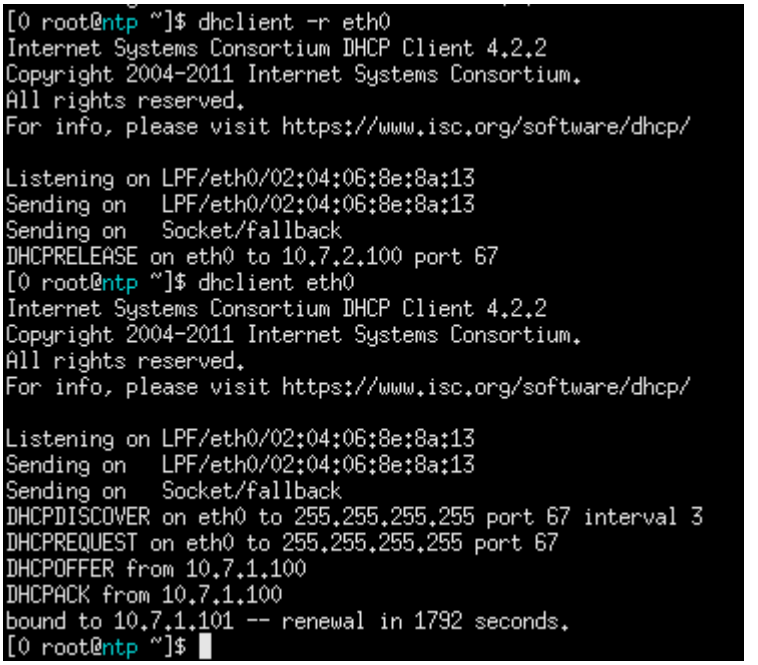
4.10 Ajoutez les lignes suivantes à la fin du fichier de configuration dhcp puis redémarrez le service dhcpd :





Le service a bien été redémarré.

I 4.11 Renouvelez le bail sur le serveur ntp (dhclient -r puis dhclient) et vérifiez qu'il obtient bien l'IP 10.N.1.101.





il obtient bien l'adresse 10.7.1.101 pour l'interface eth0.

Q 4.4 Cette méthode d'identification (avec l'adresse MAC) est-elle sûre ? Autrement dit est-ce qu'un autre hôte peut obtenir l'IP 10.N.1.101 auprès du serveur dhcp ?

L'identification par adresse MAC n'est pas sûre. Même si l'adresse MAC est unique "en dur", elle peut être très facilement modifiée de manière logicielle (Le MAC Spoofing).

Si un autre hôte sur le réseau usurpe l'adresse MAC du client légitime en modifiant sa propre configuration, le serveur DHCP ne verra aucune différence et lui attribuera l'adresse IP 10.N.1.101.

Exercice 5 — Configuration du serveur NTP

Objectif : L'objectif de cet exercice est de configurer un serveur NTP (Network Time Protocol) afin de permettre la synchronisation de l'heure sur les différentes machines du réseau. La synchronisation du temps est essentielle dans un réseau informatique, notamment pour la cohérence des journaux, la sécurité et le bon fonctionnement de certains services. Cet exercice vise également à intégrer le serveur NTP dans la configuration DHCP afin que les clients reçoivent automatiquement l'adresse du serveur de temps. Il permet ainsi de comprendre l'importance des options DHCP avancées dans la distribution des paramètres réseau.

Étapes réalisées :

- **Option DHCP pour le temps** : Ajout de l'option ntp-servers dans la configuration du serveur DHCP.

- **Configuration du serveur NTP** : Modification du fichier /etc/ntp.conf pour définir le serveur comme source de temps locale (stratum 2).

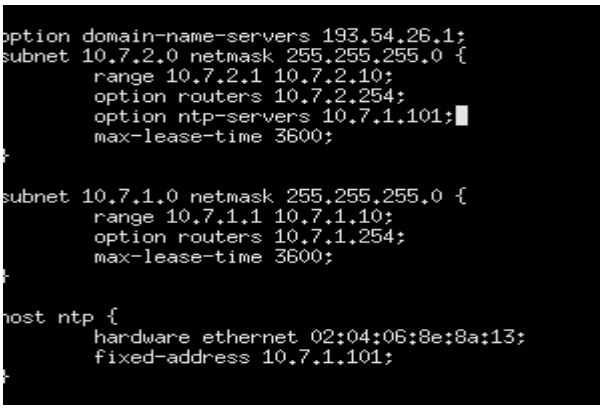
- **Synchronisation du client** : Redémarrage du service NTP sur le client Anna et vérification de la synchronisation temporelle.

- **Vérification du trafic NTP** : Analyse Wireshark des échanges entre Anna et le serveur de temps.

Commandes utilisées : man, date -s, restart, date

I 5.1 Recherchez dans le manuel des options dhcp (commande man dhcp-options), l'option permettant d'indiquer un serveur ntp aux clients qui en font la demande

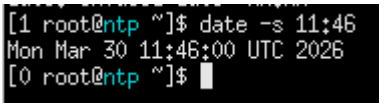
I 5.2 Ajoutez cette ligne dans fichier de configuration du service dhcp, à l'intérieur de la déclaration du réseau 10.N.2.0



I 5.3 Redémarrez le service dhcpd.



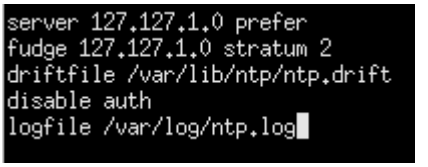
5.4 Sur ntp, changez l'heure avec la commande date en ajoutant dix minutes à l'heure actuelle :



5.5 Supprimez puis recréez le fichier /etc/ntp.conf.



I 5.6 Éditez avec nano le fichier /etc/ntp.conf pour avoir le contenu suivant :



5.7 Redémarrez le serveur ntp

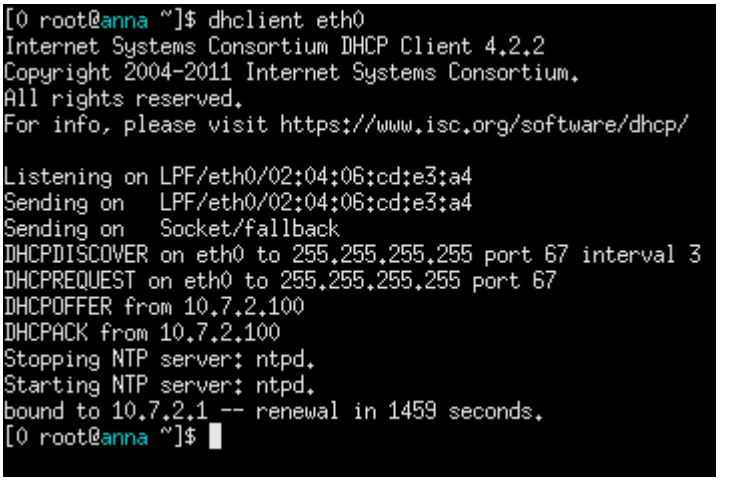


Sur Anna :
5.8 Redémarrez le serveur ntp. (Il faut aussi qu'un serveur ntp soit en exécution sur anna afin qu'il se synchronise périodiquement avec celui de l'hôte ntp.)

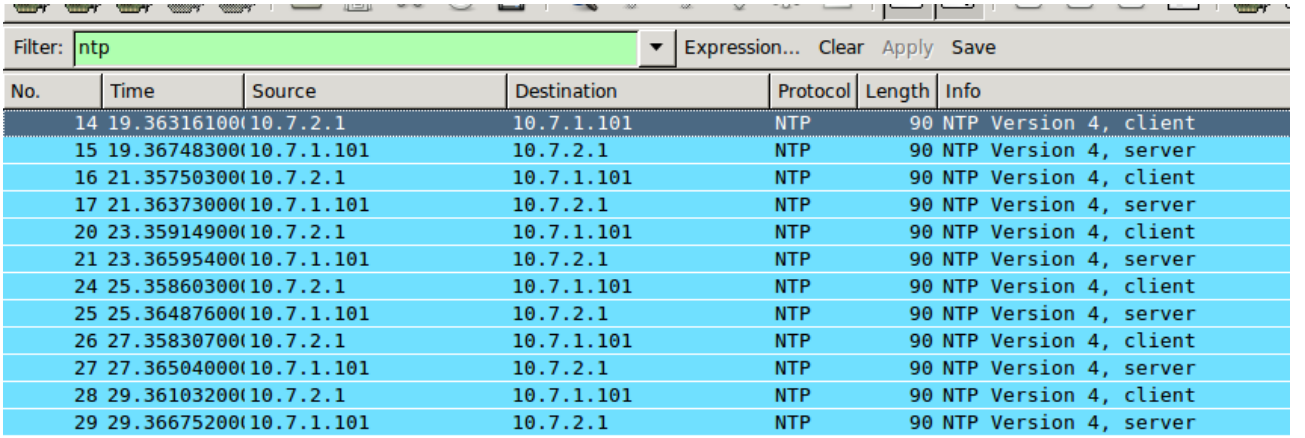


I 5.9 Démarrez une capture de trame avec wireshark.

I 5.10 Renouvelez le bail.



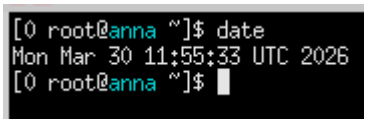
I 5.11 Arrêtez la capture puis entrez ntp comme filtre



I 5.12 Vérifiez que la capture fait bien apparaître les requêtes NTP d’anna ainsi que les réponses du serveur ntp.

La capture d'écran confirme bien l'échange entre le client et le serveur de temps grâce au protocole NTP Version 4. On observe une série de requêtes émises par l'hôte d'Anna (10.7.2.1), identifiées par le flag "client" dans la colonne Info, suivies immédiatement des réponses du serveur (10.7.1.101) marquées comme "server". Cette alternance régulière de paquets à intervalles de deux secondes environ prouve que la synchronisation temporelle est active et que le client reçoit bien les données nécessaires pour mettre son horloge à jour.

I 5.13 Sur anna, affichez la date



On a bien l’heure + 10 min d’avance, ntp et anna sont bien synchronisés.

Exercice 6 — Configuration du client

Objectif : L’objectif de cet exercice est d’étudier et de modifier le comportement du client DHCP en personnalisant les paramètres qu’il envoie au serveur lors de la demande d’adresse IP. Cela permet de mieux comprendre le fonctionnement interne du protocole DHCP, notamment les échanges entre le client et le serveur lors de l’attribution d’un bail. Cet exercice vise également à expérimenter différentes options de configuration, comme la demande d’une adresse IP spécifique ou la définition de certains paramètres réseau, afin d’observer leur impact sur la configuration obtenue.

Étapes réalisées :

- **Personnalisation des requêtes client** : Modification du fichier `/etc/dhcp/dhclient.conf` pour forcer des paramètres spécifiques.

- **Demandes spécifiques** : Configuration pour demander une IP précise (10.7.2.8), un bail d'une demi-heure et exiger la présence d'un routeur dans l'offre.

- **Application des paramètres** : Renouvellement du bail pour valider la prise en compte des options send et require.

On peut aussi configurer les clients pour paramétrer les requêtes qu’ils enverront au serveur dhcp. Le fichier de configuration du client dhcp est /etc/dhcp/dhclient.conf. En regardant le manuel de ce fichier ou en recherchant sur internet, trouvez (puis testez) comment faire en sorte que :

- le client demande une IP particulière au serveur (par exemple, si anna veut toujours utiliser 10.N.2.8) ;

- le client demande une longueur de bail particulière (par exemple une demi-heure) ;

- le client rejette une offre ne contenant pas une option particulière (par exemple, rejeter une offre qui ne contient pas l’adresse d’un routeur).

On utilise les options suivantes :

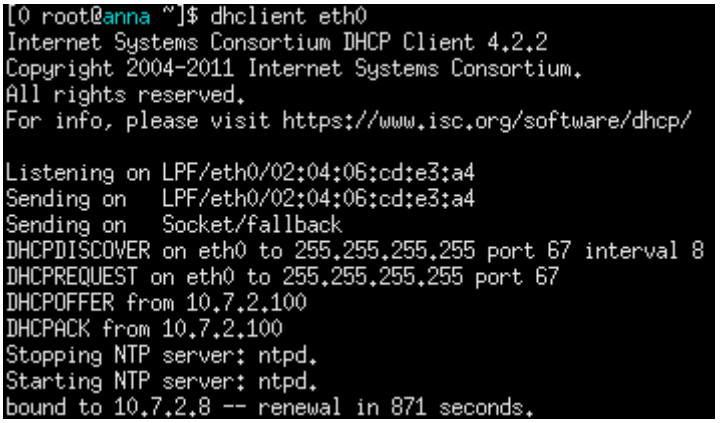
- send dhcp-requested-address 10.7.2.8;

- send dhcp-lease-time 1800;

- require routers;



Après avoir fait dhclient -r eth0, on redemande un bail avec dhclient eth0



Dans ces options, anna demande donc comme adresse : 10.7.2.8 obligatoirement, la demande de bail anna demande pour 1800 secondes soit 30 mins et enfin la dernière option requiert qu’il faut obligatoire une adresse routeur comme passerelle.

Conclusion :

Ce premier TP m’a permis de comprendre le fonctionnement général du service DHCP et son rôle essentiel dans l’attribution automatique des adresses IP au sein d’un réseau. J’ai appris à configurer manuellement des interfaces réseau, à mettre en place un serveur DHCP ainsi qu’un relai DHCP permettant la communication entre plusieurs sous-réseaux. J’ai également découvert comment ajouter des options supplémentaires, comme l’attribution d’adresses IP fixes et la configuration d’un serveur NTP pour la synchronisation de l’heure des machines. L’ensemble de ces manipulations m’a permis de mieux comprendre l’organisation d’un réseau et les interactions entre les différents équipements.

Au cours de ce TP, certaines difficultés ont été rencontrées, notamment lors de la configuration des adresses IP et du fichier de configuration du serveur DHCP. Des erreurs de syntaxe ou des oublis dans les fichiers pouvaient empêcher le bon fonctionnement du service. Pour résoudre ces problèmes, j’ai vérifié attentivement les commandes saisies, relu les fichiers de configuration et redémarré les services après chaque modification. L’utilisation d’outils de test comme la commande ping ou l’analyse des messages DHCP a également permis d’identifier et de corriger les erreurs plus facilement.

Ce TP est particulièrement utile dans un contexte professionnel, car la configuration d’un service DHCP est une tâche courante dans l’administration des réseaux informatiques. Les compétences acquises lors de ce TP, comme la configuration d’adresses IP, la gestion de services réseau et l’analyse des communications entre machines, sont essentielles pour un technicien réseaux et télécommunications. Ce travail constitue donc une première étape importante vers la maîtrise des services réseaux utilisés dans les infrastructures professionnelles.

GROT Yanis GROUPE 3

TP2 : Le service SSH

Table des matières :

1. Introduction
2. Exercice 1 : Création du réseau et configuration des interfaces
3. Exercice 2 : Utilisation de telnet
4. Exercice 3 : Utilisation de la commande ssh
5. Exercice 4 : Utilisation de la commande scp
6. Exercice 5 : Observation du mécanisme de chiffrement
7. Exercice 6 : Authentification par clé
8. Exercice 7 : Création d’un tunnel ssh
9. Conclusion

Introduction :

Ce TP porte sur le protocole SSH (*Secure Shell*), un protocole de communication sécurisé largement utilisé dans l'administration des systèmes et réseaux informatiques. Pour illustrer son intérêt, nous commencerons par étudier Telnet, un protocole plus ancien qui transmet les données en clair sur le réseau, afin de mettre en évidence les risques que SSH vient corriger. Les manipulations sont réalisées dans un environnement de réseau virtuel simulé avec l'outil Marionnet, sur une machine utilisant une distribution Linux Debian.

L'objectif principal de ce TP est de se familiariser avec les fonctionnalités du protocole SSH, principalement du côté client, ainsi qu'avec les mécanismes de chiffrement qu'il met en œuvre. Plus concrètement, il s'agit de comprendre comment SSH garantit la confidentialité des échanges, d'apprendre à l'utiliser pour se connecter à distance, transférer des fichiers et automatiser des tâches, et enfin de mettre en pratique la notion de tunnel SSH pour sécuriser un trafic non chiffré.

Le TP se déroule en plusieurs étapes progressives. Après avoir créé et configuré le réseau de travail, nous observerons la vulnérabilité de Telnet par une capture de trafic. Nous explorerons ensuite les commandes ssh et scp pour la connexion distante et le transfert sécurisé de fichiers. Nous étudierons également les mécanismes de chiffrement asymétrique utilisés par SSH, notamment la gestion des clés publiques et privées, avant de mettre en place une authentification par clé permettant de se connecter sans mot de passe. Enfin, nous créerons un tunnel SSH pour sécuriser une communication HTTP entre deux hôtes du réseau.

Exercice 1 — Création du réseau et configuration des interfaces

Objectif : Mettre en place la topologie réseau du TP en configurant les quatre hôtes (admin, spy, ssh1, ssh2) avec leurs adresses IP statiques et vérifier la connectivité entre eux.

Démarche : On a créé un nouveau projet dans marionnet et ajouté les quatre hôtes (admin, spy, ssh1, ssh2), un hub H et un switch S en respectant le câblage de la topologie. On a ensuite démarré tous les équipements et ouvert une session root sur chacun. On a attribué les adresses IP statiques sur chaque hôte puis vérifié la connectivité avec des pings entre tous les hôtes.

Étapes réalisées :

- Création d'un nouveau projet dans Marionnet.

- Ajout des machines admin, spy, ssh1 et ssh2.

- Connexion des équipements avec un hub et un switch conformément au schéma du TP.

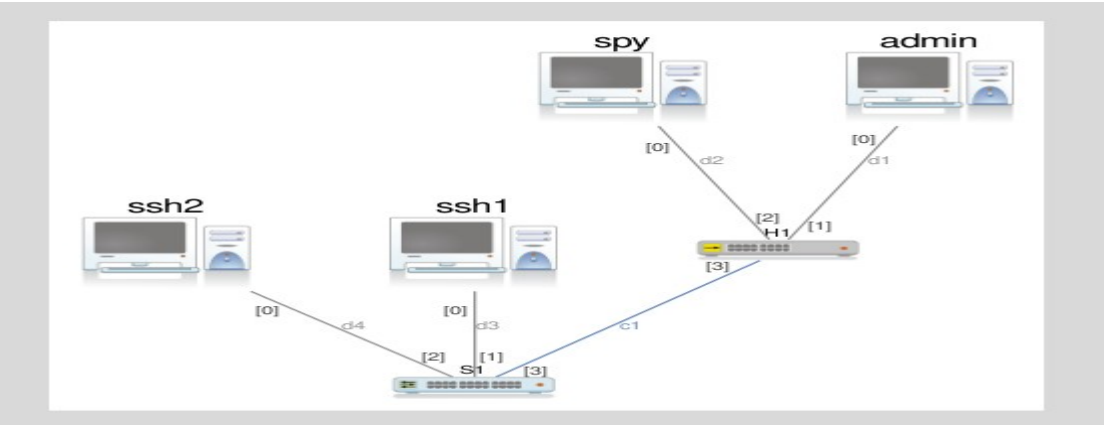
- Démarrage des machines puis ouverture d'une session root.

- Configuration manuelle des adresses IP sur chaque interface réseau.

- Vérification de la connectivité avec la commande ping.

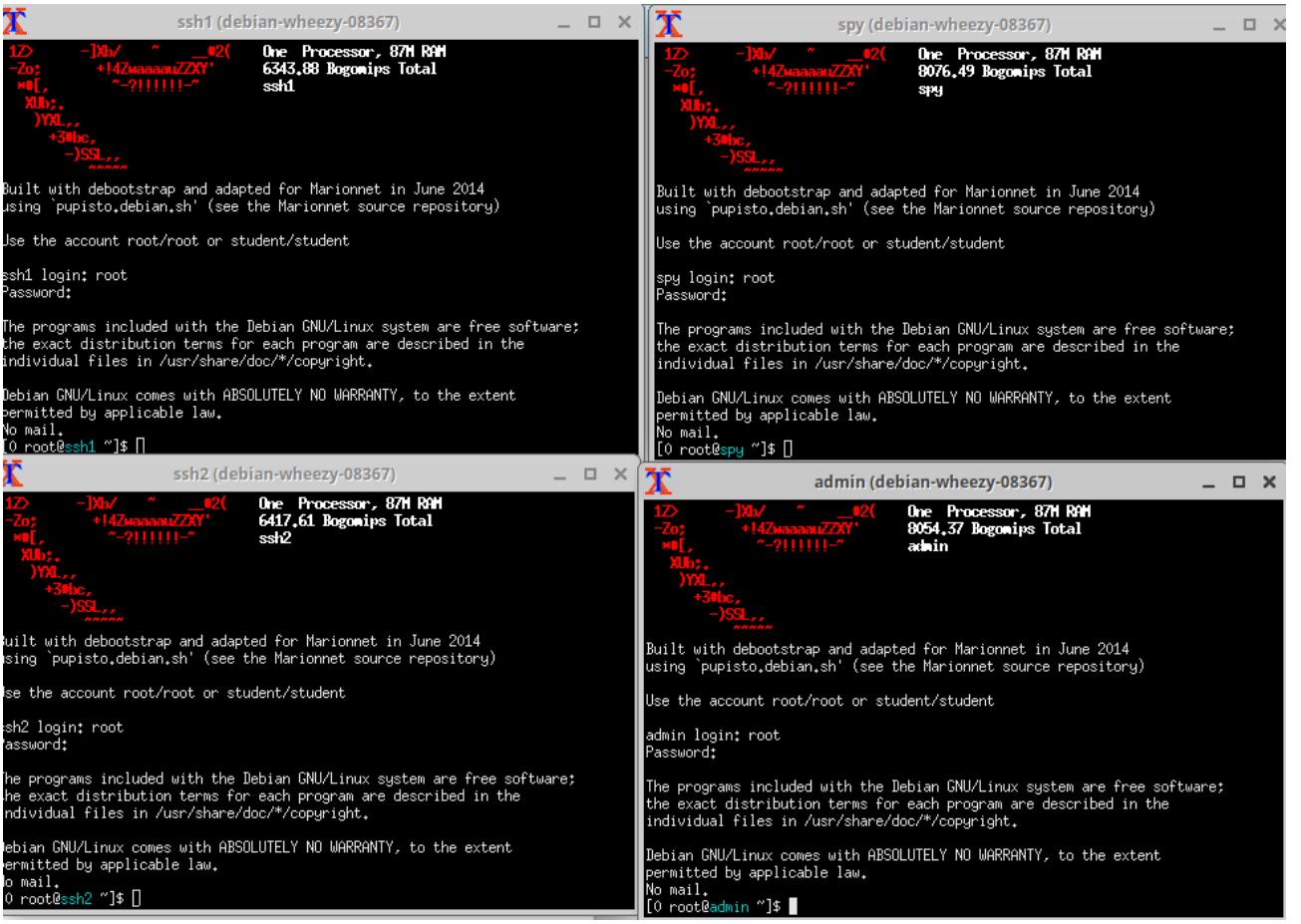
Commandes utilisées : ip addr, ip link set eth0, ping

I 1.2 Ajoutez les équipements pour avoir le réseau de la figure en introduction. H est un hub et S un switch. (Attention au type de câble . . .)



Nous avons bien créé la configuration sur marionnet.**1.3 Démarrez tous les équipements**

I 1.4 Ouvrez une session root (mot de passe=root) sur chaque hôte



I 1.5 À l'aide de la commande ip (voir exercice 2 du TP 1), attribuez les IP aux interfaces

hôtes

admin : 10.4.0.1

spy : 10.4.0.2

ssh1 : 10.4.0.100

ssh2 : 10.4.0.101

Pour admin :

```
[0 root@admin ~]$ ip addr add 10.4.0.1/24 dev eth0
[0 root@admin ~]$ ip link set eth0 up
[0 root@admin ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:04:c8:61
          inet addr:10.4.0.1  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe04:c861/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:328 (328.0 B)
          Interrupt:5
```

Pour spy :

```
[0 root@spy ~]$ ip addr add 10.4.0.2/24 dev eth0
[0 root@spy ~]$ ip link set eth0 up
[0 root@spy ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:ea:ba:1a
          inet addr:10.4.0.2  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:feea:ba1a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:402 (402.0 B)  TX bytes:328 (328.0 B)
          Interrupt:5
```

Pour ssh1 :

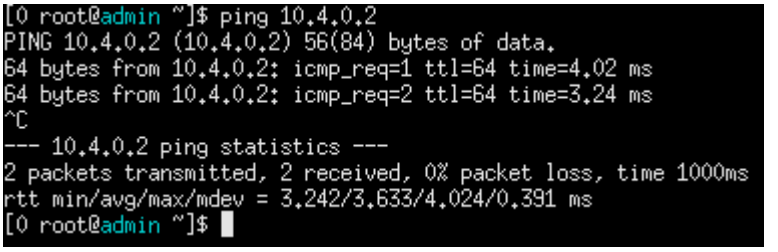
```
[0 root@ssh1 ~]$ ip addr add 10.4.0.100/24 dev eth0
[0 root@ssh1 ~]$ ip link set eth0 up
[0 root@ssh1 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:1d:70:06
          inet addr:10.4.0.100  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe1d:7006/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:15 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:822 (822.0 B)  TX bytes:238 (238.0 B)
          Interrupt:5
```

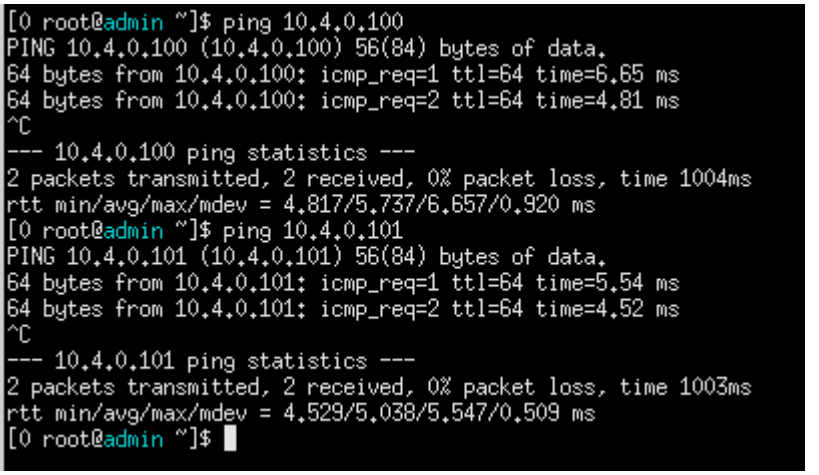
Pour ssh2 :

```
[0 root@ssh2 ~]$ ip addr add 10.4.0.101/24 dev eth0
[0 root@ssh2 ~]$ ip link set eth0 up
[0 root@ssh2 ~]$ ifconfif eth0
-bash: ifconfif: command not found
[127 root@ssh2 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:8a:77:6e
          inet addr:10.4.0.101  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe8a:776e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:24 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1260 (1.2 KiB)  TX bytes:468 (468.0 B)
          Interrupt:5
```

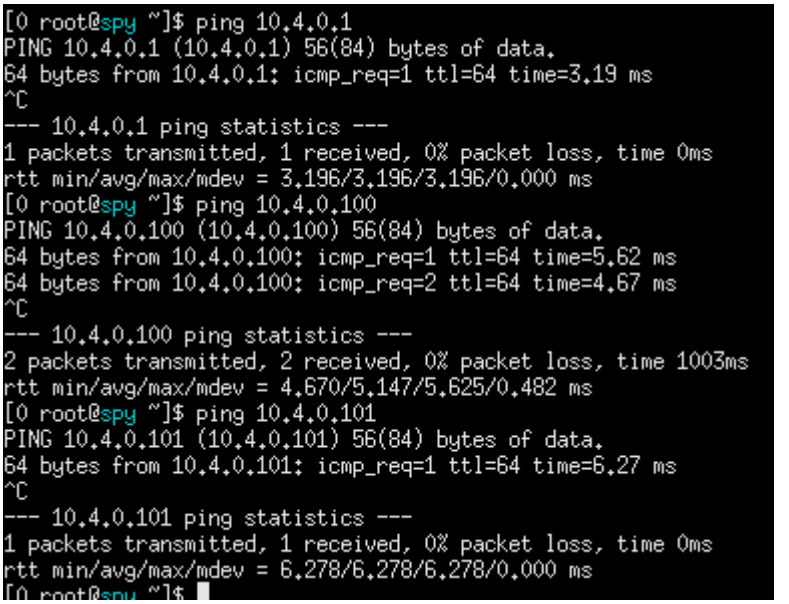
I 1.6 Vérifiez que les hôtes peuvent bien s’échanger des messages ping.

Pour admin :

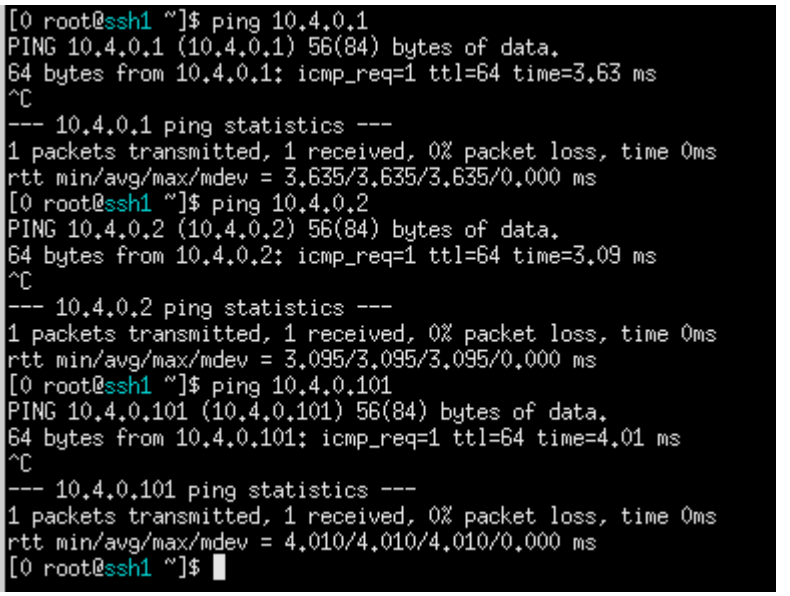




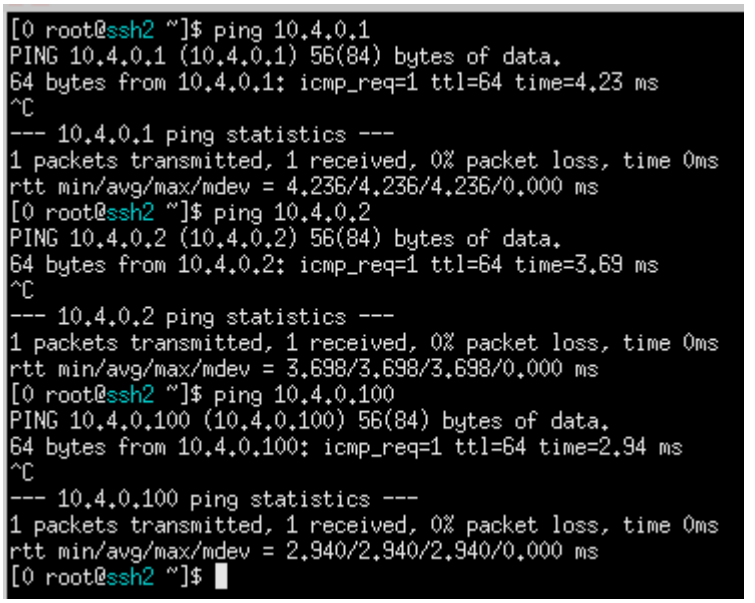
Pour spy :



Pour ssh1 :



Pour ssh2 :



Tous les pings fonctionnent correctement.

Exercice 2 — Utilisation de telnet

Objectif : Mettre en évidence la faille de sécurité de telnet en montrant que le mot de passe transite en clair sur le réseau et peut être intercepté par un tiers.

Démarche : On a activé telnet sur ssh1 en éditant /etc/inetd.conf et en retirant #<off># devant la ligne telnet. On a démarré inetd, puis lancé une capture wireshark sur spy pendant qu'admin ouvrait une session telnet sur ssh1. Sur spy, on a analysé le flux TCP avec Analyze Follow TCP Stream et observé le mot de passe en clair.

Étapes réalisées :

- Activation du service Telnet dans le fichier /etc/inetd.conf

- Redémarrage du service inetd

- Vérification de l’exécution du processus telnetd

- Capture du trafic réseau avec Wireshark sur la machine spy

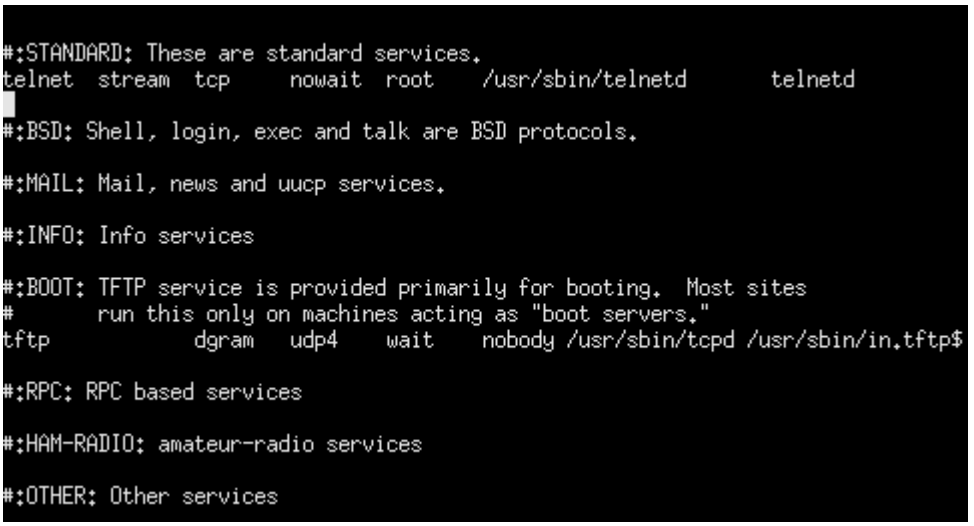
- Connexion Telnet depuis admin vers ssh1

- Analyse du flux TCP capturé

Commandes utilisées : nano, restart, ps aux, telnet

- Aucun problème rencontré. Le mot de passe est bien apparu en clair dans la capture, ce qui confirme la vulnérabilité de telnet.

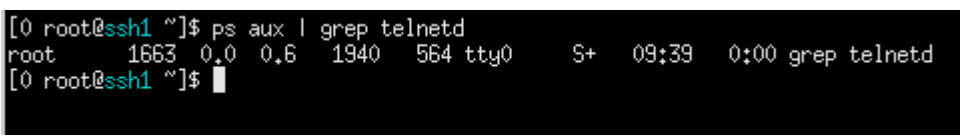
I 2.1 Éditez le fichier /*etc*/inetd.conf afin d’activer telnet. Il faut pour cela retirer les caractères #<off># se trouvant devant le mot telnet



2.2 Démarrez le service inetd :

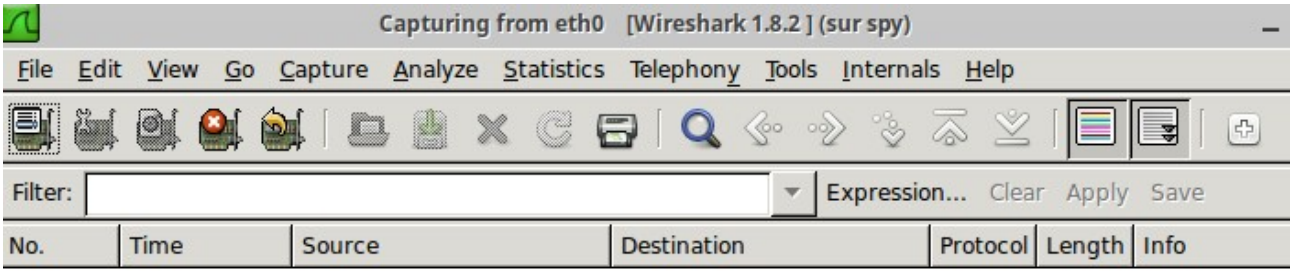


I 2.3 Exécutez la commande suivante pour voir si il y a un processus nommé telnetd :

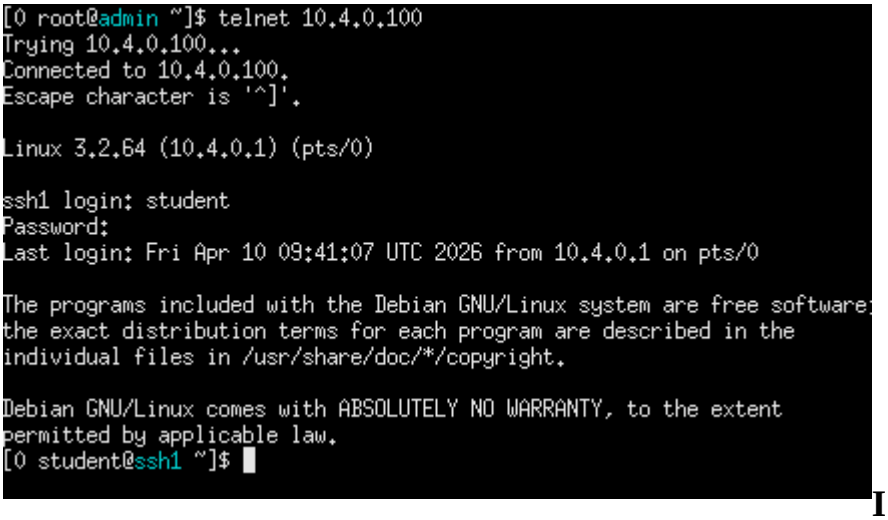


Oui, il y a un processus telnetd qui est bien lancé.

I 2.4 Sur spy : lancez wireshark et démarrez une capture sur eth0



I 2.5 Sur admin : ouvrez une session telnet vers ssh1 :



I 2.6 Sur spy : stoppez la capture, puis lancez l'analyse de flot TCP (menu Analyze → Follow TCP Stream). Vous devriez voir le mot de passe en clair.

```
u.ud.de.en.nt.t... .... .... .... .... .d.de.en.nt.t... .... .... .... .e.
..
Password: student
.
.Last login: Fri Apr 10 09:44:06 UTC 2026 from 10.4.0.1 on pts/0
.
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
.permitted by applicable law.
.[0 student@.[0;36mssh1.[m ~]$
```

Q 2.1 Est-ce que l'utilisation d'un switch à la place d'un hub aurait résolu le problème (i.e., si H était un switch est-ce que spy aurait tout de même pu capturer le mot de passe) ? Justifiez.

Non, utiliser un switch n'aurait pas résolu le problème. Un hub diffuse toutes les trames à tous ses ports, ce qui permet à spy de voir le trafic entre admin et ssh1. Un switch, lui, n'envoie les trames qu'au port destinataire, donc en théorie spy ne verrait pas passer le trafic.

Mais dans notre topologie, spy et admin sont tous les deux connectés au même hub H. Même si H était un switch, spy reste sur le même segment réseau qu'admin, et pourrait toujours recevoir les trames échangées entre admin et ssh1, car elles passent forcément par H avant d'atteindre le switch S puis ssh1.

I 2.7 Sur ssh1 : refaites le test de l'instruction I 2.3.

```
[0 root@ssh1 ~]$ ps aux | grep telnetd
root      1663  0.0  0.6   1940   564 tty0     S+   09:39   0:00 grep telnetd
[0 root@ssh1 ~]$ ps aux | grep telnetd
root      1705  0.0  1.6   4724  1484 ?        S    09:47   0:00 telnetd
root      1725  0.0  0.6   1940   568 tty0     S+   09:50   0:00 grep telnetd
[0 root@ssh1 ~]$
```

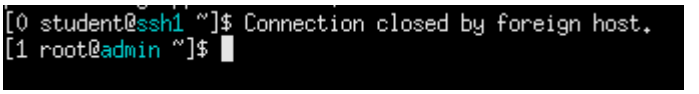
Cette fois-ci, nous voyons 2 programmes lancés. Un sur ssh1 comme celui de la question 2.3 et un autre lancé sur la machine admin connecté à ssh1.

Q 2.2 Quel est, à votre avis, l'intérêt d'inetd ?

inetd est un service qui écoute sur plusieurs ports à la fois et qui lance les services uniquement à la demande, c'est-à-dire quand une connexion arrive. Sans inetd, chaque service devrait tourner en permanence en arrière-plan, ce qui consommerait inutilement de la mémoire et des ressources.

Avec inetd, un seul processus gère l'écoute pour tous ces services, et ne lance le bon service que lorsqu'un client se connecte. C'est donc une solution plus économe en ressources.

I 2.8 Sur admin : fermez la session telnet (ctrl+d ou exit)



I 2.9 Sur spy : fermez wireshark

Exercice 3 — Utilisation de la commande ssh

Objectif : Se familiariser avec les différentes utilisations de la commande ssh : connexion distante, changement d'utilisateur et exécution de commandes à distance.

Démarche : On a démarré le serveur SSH sur ssh1 et ssh2, puis effectué différentes connexions depuis admin : en root, en student, et en exécutant des commandes à distance. À la première connexion, un message indiquait que l'hôte était inconnu, on a répondu yes pour accepter et mémoriser la clé publique du serveur. La commande ls /root en student a échoué car student n'a pas les droits sur ce répertoire.

Étapes réalisées :

- Démarrage du serveur SSH sur ssh1 et ssh2.

- Première connexion SSH depuis admin

- Validation de l'empreinte de la clé du serveur

- Connexion avec différents utilisateurs

- **Exécution de commandes distantes via SSH.**

Commandes utilisées : restart, ssh

3.1 Redémarrez le serveur ssh sur ssh1 et ssh2.

Sur ssh1 :

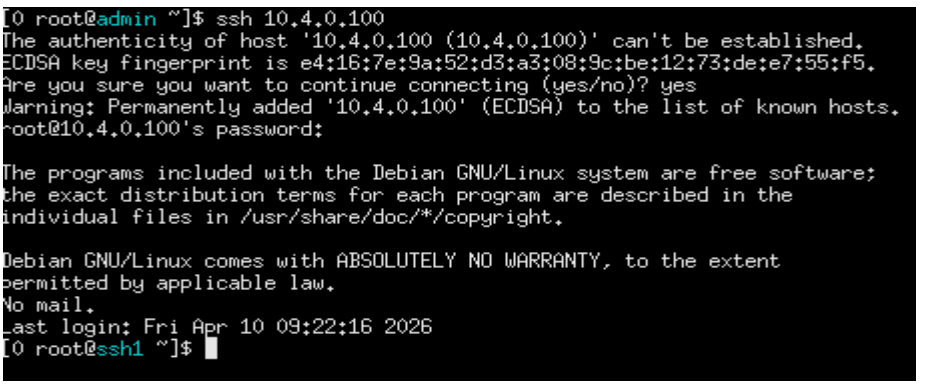


Sur ssh2 :



I 3.2 Sur admin, ouvrez une session sur ssh1 :

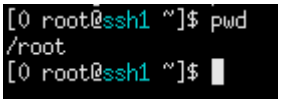
I 3.3 Entrez yes puis le mot de passe root.



Q 3.1 Quel est maintenant le message d'invite dans la fenêtre d'admin ? Pourquoi ?

Cela s'explique par le fait qu'après la connexion SSH, on n'est plus sur la machine admin mais sur la machine ssh1. Le terminal montre donc l'utilisateur courant (root) et la machine sur laquelle on est connecté (ssh1).

Q 3.2 Quel est le répertoire courant ?



Le répertoire courant est /root.

I 3.4 Utilisez la commande suivante sur admin :

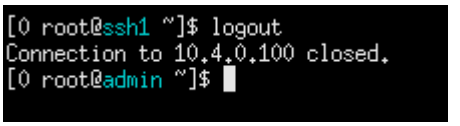


I 3.5 Vérifiez que le message s'est bien affiché dans la fenêtre de ssh1.



Le message est bien affiché sur le terminal de ssh1.

I 3.6 Quittez la session ssh ouverte sur admin (ctrl+d ou exit)



I 3.7 Sur admin, ouvrez une session student sur ssh1

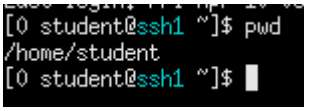


Le message n’apparaît plus car la machine connaît maintenant l’adresse IP de la machine.

Q 3.3 Quel est maintenant le message d’invite dans la fenêtre d’admin ? Pourquoi ?

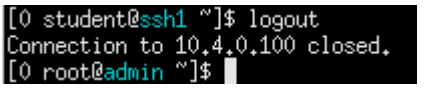
Le message d'invite est student@ssh1 ~$. On est toujours sur ssh1 mais cette fois connecté avec le compte student, donc le nom d'utilisateur change dans le prompt.

Q 3.4 Quel est le répertoire courant ?



Le répertoire courant est /*home*/student.

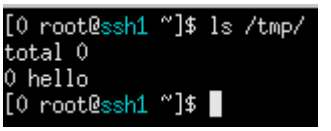
I 3.9 Quittez la session ssh ouverte sur admin (ctrl+d ou exit).



I 3.10 Sur admin, exécutez la commande ci-dessous puis entrez le mot de passe :



I 3.11 Sur ssh1, vérifiez que la commande touch a bien été exécutée.



La commande touch a bien été exécuté, le fichier hello a bien été créé dans le dossier tmp.

3.12 Sur admin, exécutez la commande ci-dessous puis entrez le mot de passe :



Q 3.5 Cette dernière commande a-t-elle fonctionné ? Pourquoi ?

Non, la commande n'a pas fonctionné. L'utilisateur student n'a pas les droits pour lire le répertoire /root, qui appartient à root et dont les permissions sont restreintes. Le système retourne donc une erreur "Permission denied".

Exercice 4 — Utilisation de la commande scp

Objectif : Apprendre à copier des fichiers et des répertoires de manière sécurisée entre deux machines en utilisant la commande scp.

Démarche : On a créé les fichiers et répertoires nécessaires sur admin, puis effectué différentes copies vers ssh1 en testant les différentes syntaxes de scp, dans les deux sens (client vers serveur et serveur vers client). On a vérifié sur ssh1 la présence des fichiers copiés et leurs propriétaires avec ls -l.

Étapes réalisées :

- Création de fichiers et répertoires sur admin

- Copie de fichiers simples et de répertoires vers ssh1

- Test de copie depuis le serveur vers le client.

Commandes utilisées : scp

I 4.1 Sur admin, créez les fichiers et répertoires suivants (dans le répertoire courant) : fic1, rep1, rep1/fic2, rep1/rep2 et rep1/rep2/fic3.

```
[0 root@admin ~]$ touch fic1
[0 root@admin ~]$ mkdir rep1
[0 root@admin ~]$ touch rep1/fic2
[0 root@admin ~]$ mkdir rep1/rep2
[0 root@admin ~]$ touch rep1/rep2/fic3
[0 root@admin ~]$
```

I 4.2 Exécutez les commandes suivantes :

```
[0 root@admin ~]$ scp fic1 10.4.0.100:
root@10.4.0.100's password:
fic1                                          100%    0     0.0KB/s   00:00
[0 root@admin ~]$ scp rep1/fic2 student@10.4.0.100:
student@10.4.0.100's password:
fic2                                          100%    0     0.0KB/s   00:00
[0 root@admin ~]$ scp rep1/rep2/fic3 10.4.0.100:/etc
root@10.4.0.100's password:
fic3                                          100%    0     0.0KB/s   00:00
[0 root@admin ~]$ scp -r rep1 10.4.0.100:
root@10.4.0.100's password:
fic2                                          100%    0     0.0KB/s   00:00
fic3                                          100%    0     0.0KB/s   00:00
[0 root@admin ~]$
```

Q 4.1 À quoi sert l'option -r utilisée pour la dernière commande :

L'option -r permet de copier un répertoire de manière récursive, c'est-à-dire en copiant le répertoire et tout son contenu (sous-répertoires et fichiers inclus).

Q 4.2 À quels emplacements ont été copiés les fichiers et répertoires sur ssh1, et quels sont leurs propriétaires ?

- fic1 - /root/fic1, propriétaire root

- fic2 - /home/student/fic2, propriétaire student (car copié avec l'identité student)

- fic3 - /etc/fic3, propriétaire root

- rep1 - /root/rep1/, propriétaire root

I 4.3 Donnez et testez sur admin les commandes permettant de :

```
[0 root@admin ~]$ scp 10.4.0.100:/etc/dhcp/dhcpd.conf .
root@10.4.0.100's password:
dhcpd.conf                                    100% 3491     3.4KB/s   00:00
[0 root@admin ~]$
```

```
[0 root@admin ~]$ scp 10.4.0.100:/etc/dhcp/dhcpd.conf .
root@10.4.0.100's password:
dhcpd.conf                                    100% 3491     3.4KB/s   00:00
[0 root@admin ~]$ scp -r student@10.4.0.101:/home/student /tmp/
The authenticity of host '10.4.0.101 (10.4.0.101)' can't be established.
ECDSA key fingerprint is e4:16:7e:9a:52:d3:a3:08:9c:be:12:73:de:e7:55:f5.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.4.0.101' (ECDSA) to the list of known hosts.
student@10.4.0.101's password:
.bash_aliases                                 100% 2383     2.3KB/s   00:00
.profile                                      100%  675     0.7KB/s   00:00
.bash_logout                                  100%  220     0.2KB/s   00:00
.bashrc                                       100% 3392     3.3KB/s   00:00
authorized_keys                               100%  396     0.4KB/s   00:00
[0 root@admin ~]$
```

Exercice 5 — Observation du mécanisme de chiffrement

Objectif : Comprendre le fonctionnement des clés SSH et observer les conséquences d'un changement de clé côté serveur sur la connexion du client.

Démarche :On a listé les clés SSH sur ssh1 avec ls -l /etc/ssh et observé leurs droits. On a ensuite régénéré la clé ecdsa avec ssh-keygen pour simuler un changement de clé. En tentant de se reconnecter depuis admin, un message WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED est apparu. On a supprimé l'ancienne clé mémorisée avec ssh-keygen -R puis reconnecté normalement.

Étapes réalisées :

- Observation des clés SSH dans /etc/ssh.

- Génération d'une nouvelle paire de clés ECDSA.

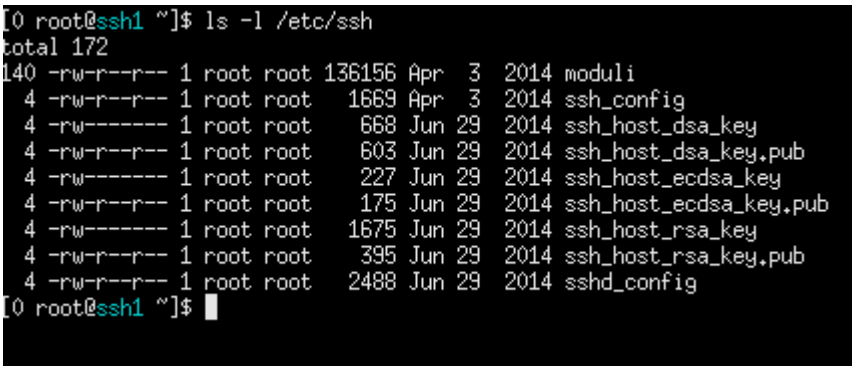
- Tentative de reconnexion depuis admin.

- Analyse de l'alerte de sécurité générée par SSH

- Suppression de l'ancienne clé connue.

Commandes utilisées : ls -l, ssh-keygen

I 5.1 Sur ssh1, listez le contenu du répertoire /etc/ssh en utilisant l'option -l.



Q 5.1 Quels sont les algorithmes de chiffrement supportés ?

Les algorithmes supportés sont dsa, ecdsa et rsa, visibles grâce aux fichiers ssh_host_dsa_key, ssh_host_ecdsa_key et ssh_host_rsa_key présents dans /etc/ssh.

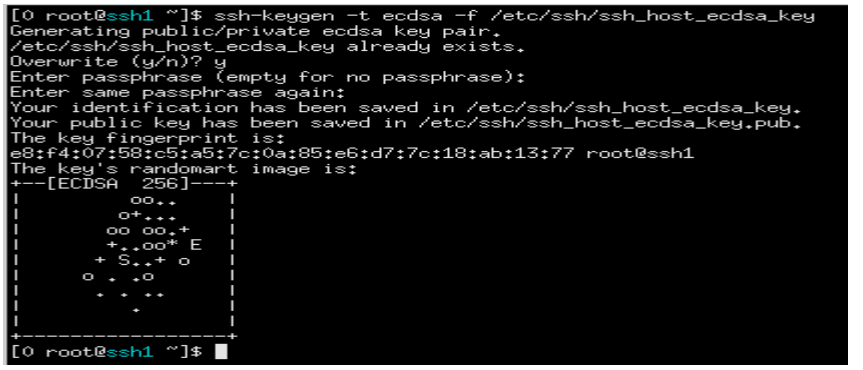
Q 5.2 Quels sont les droits associés aux fichiers contenant les clés privées ? Et à ceux contenant les clés publiques ? Pourquoi ?

Les clés privées (ssh_host_dsa_key, ssh_host_ecdsa_key, ssh_host_rsa_key) ont les droits 600, c'est-à-dire lecture et écriture pour root uniquement, et aucun droit pour les autres utilisateurs.

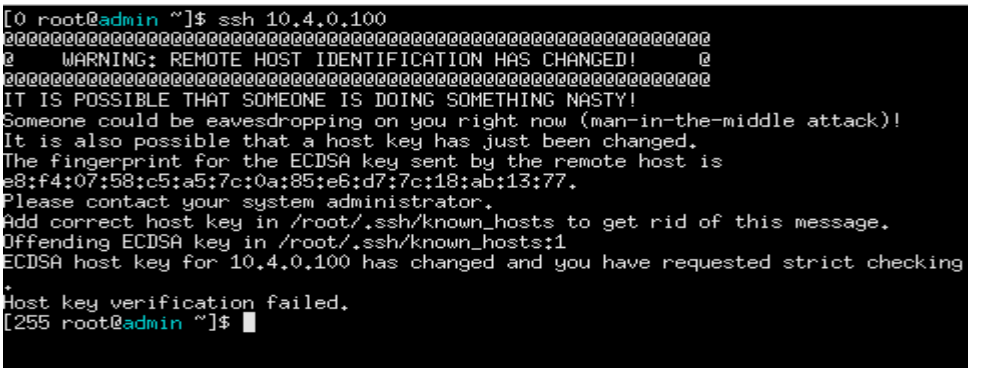
Les clés publiques (fichiers .pub) ont les droits 644, c'est-à-dire lisibles par tous mais modifiables uniquement par root.

C'est une mesure de sécurité: la clé privée doit rester secrète car si elle était accessible à d'autres utilisateurs, ils pourraient s'en servir pour usurper l'identité du serveur ou se connecter sans mot de passe.

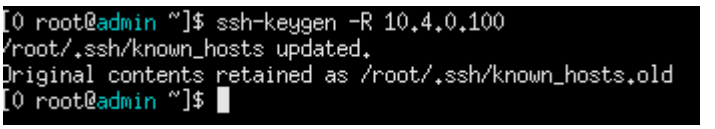
I 5.2 Sur ssh1, générez une nouvelle paire de clés pour le service ssh :



I 5.3 Sur admin, ouvrez une session sur ssh1.



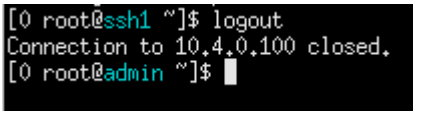
I 5.4 Supprimez la clé publique de 10.N.0.100 :



I 5.5 Retentez une connexion ssh sur 10.N.0.100 pour vérifier que c’est maintenant possible



I 5.6 Fermez la session ssh.



Exercice 6 — Authentification par clé

Objectif : Mettre en place l'authentification par clé publique pour se connecter sans mot de passe et l'exploiter dans un script bash automatisé.

Démarche: On a généré une paire de clés sur admin avec ssh-keygen en laissant le passphrase vide. On a copié la clé publique dans le fichier authorized_keys de ssh1 et ssh2 avec scp. On a vérifié que la connexion s'établissait sans mot de passe. On a ensuite créé et testé le script test-dhcp.sh qui vérifie l'état du service dhcpd sur les deux serveurs.

Étapes réalisées :

- Génération d’une paire de clés RSA.

- Copie de la clé publique sur ssh1 et ssh2.

- Connexion SSH sans mot de passe.

- Création d’un script utilisant SSH automatiquement.

Commandes utilisées : ssh-keygen, scp, ssh

I 6.1 Sur admin, générez une paire de clés :

```
[0 root@admin ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
8f:ee:0c:64:cd:90:f5:86:9b:51:8b:47:be:52:d3:59 root@admin
The key's randomart image is:
+--[ RSA 2048]----+
|        . o   E  |
|       o B o o   |
|      o + O o    |
|       + B o     |
|      o S .      |
|     o   +       |
|      . . .      |
|       +         |
|        .+       |
+-----------------+
[0 root@admin ~]$
```

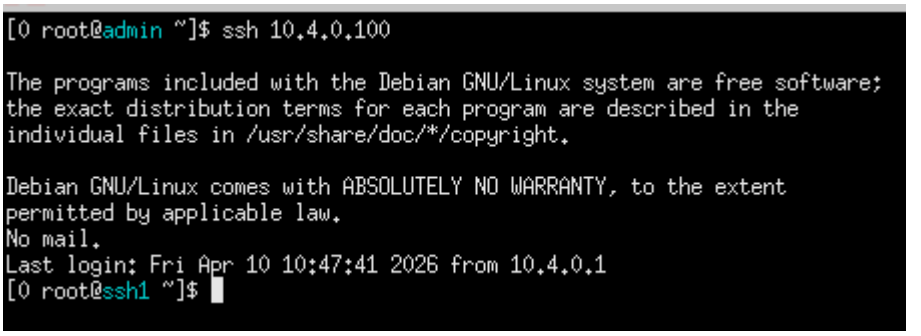
I 6.2 Avec scp copiez la clé publique /root/.ssh/id_rsa.pub dans /root/.ssh/authorized_keys sur ssh1.

```
[0 root@admin ~]$ scp /root/.ssh/id_rsa.pub 10.4.0.100:/root/.ssh/authorized_key
s
root@10.4.0.100's password:
id_rsa.pub                                    100%  392     0.4KB/s   00:00
[0 root@admin ~]$
```

I 6.3 Faites de même sur ssh2.

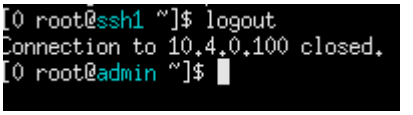
```
[1 root@admin ~]$ scp /root/.ssh/id_rsa.pub 10.4.0.101:/root/.ssh/authorized_key
s
root@10.4.0.101's password:
id_rsa.pub                                    100%  392     0.4KB/s   00:00
[0 root@admin ~]$
```

I 6.4 Ouvrez une session ssh sur ssh1.

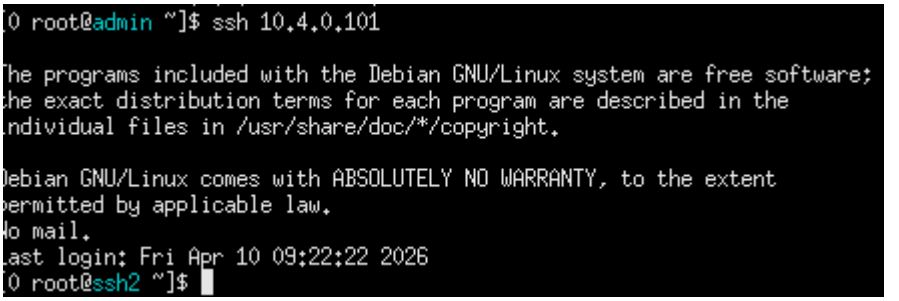


La connexion s'est fait directement car nous avons rien rentré dans les passphrases. Pas besoin de noter le mot de passe.

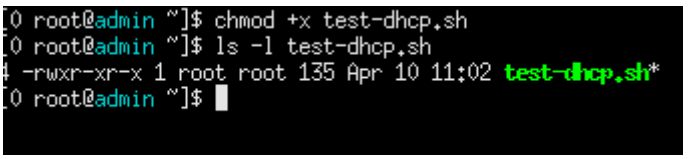
I 6.5 Fermez la session.



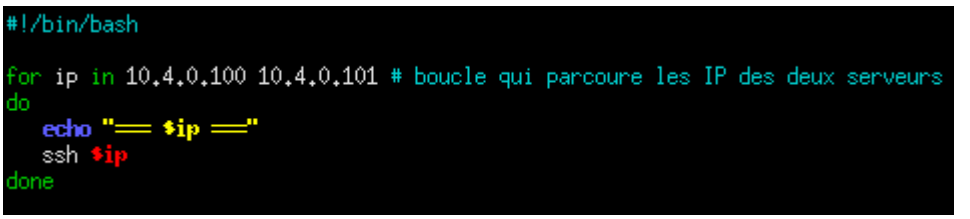
I 6.6 Suivez à nouveau les deux instructions précédentes mais cette fois-ci avec ssh2



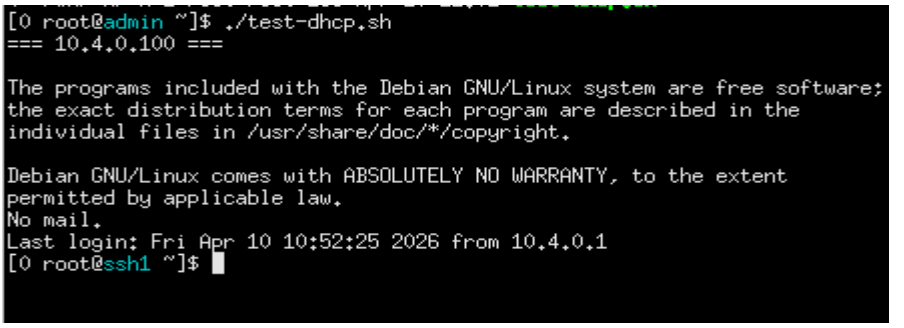
Pas besoin de mot de passe aussi, la connexion s'est faite directement.**I 6.7 Créez un fichier test-dhcp.sh et rendez le exécutable**



I 6.8 Éditez le avec nano pour avoir le contenu donné plus haut et complétez les blancs.



I 6.9 Testez votre script.



Le script a bien fonctionné tout se lance comme prévu.

Exercice 7 — Création d’un tunnel ssh

Objectif : Comprendre et mettre en place un tunnel SSH pour chiffrer un trafic HTTP normalement non sécurisé et vérifier son efficacité avec wireshark.

Démarche : On a démarré apache2 sur ssh2 puis lancé une capture wireshark sur spy. Sans tunnel, en accédant à http://10.N.0.101/ depuis epiphany, spy capturait le trafic HTTP en clair et le contenu HTML était lisible. On a ensuite créé un tunnel SSH depuis admin vers ssh1 pointant vers le port 80 de ssh2, puis accédé à http://localhost:10000. Cette fois, spy ne voyait plus que du trafic SSH chiffré, illisible.

Étapes réalisées :

- Démarrage du serveur web Apache sur ssh2

- Capture du trafic HTTP sans tunnel.

- Mise en place d’un tunnel SSH entre admin et ssh1

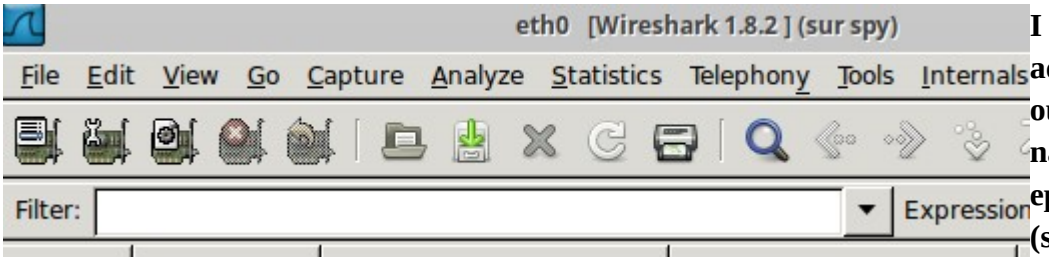
- Nouvelle capture réseau avec tunnel actif.

- Comparaison des paquets capturés.

I 7.1 Démarrez le service apache2 (un serveur web) sur ssh2.

```
[0 root@ssh2 ~]$ /etc/init.d/apache2 start
[....] Starting web server: apache2apache2: Could not reliably determine the ser
ver's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
[0 root@ssh2 ~]$
```

I 7.2 Démarrez une capture avec wireshark sur spy



I 7.3 Sur admin, ouvrez le navigateur epiphany (sans oublier l’esperluette pour que le terminal vous rende la main) :

I 7.4 Dans epiphany, entrez l’URL http://10.N.0.101/ puis validez.

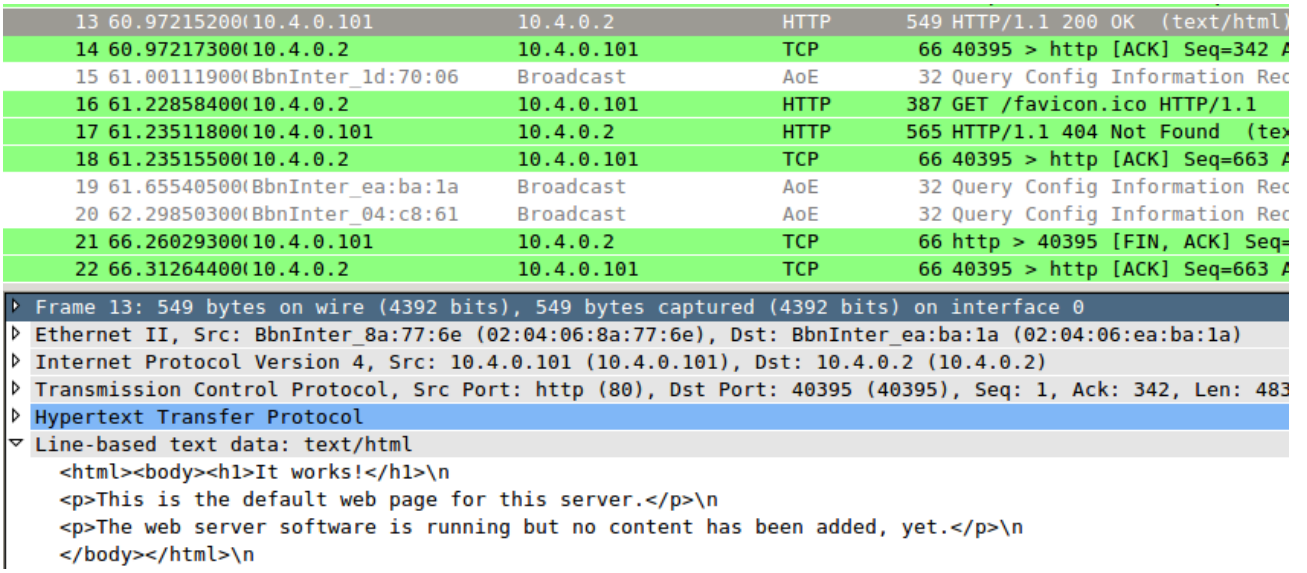


It works!

This is the default web page for this server.

The web server software is running but no content has been added, yet.

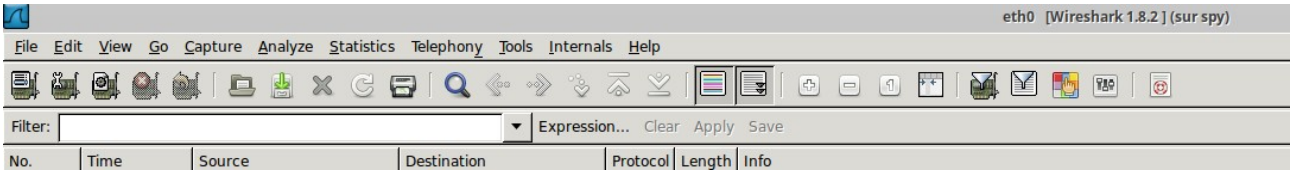
I 7.5 Arrêtez la capture sur spy et trouvez un paquet contenant du code HTML envoyé par ssh2. (Le code devrait apparaître dans le panneau du bas, celui qui fait apparaître la trame sous forme brute.)



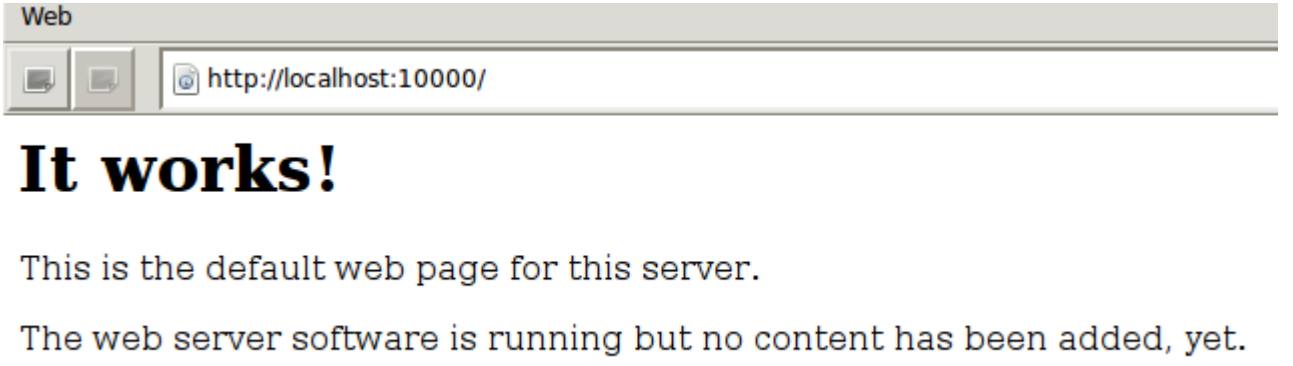
I 7.6 Sur admin, lancez la commande ssh créant le tunnel :



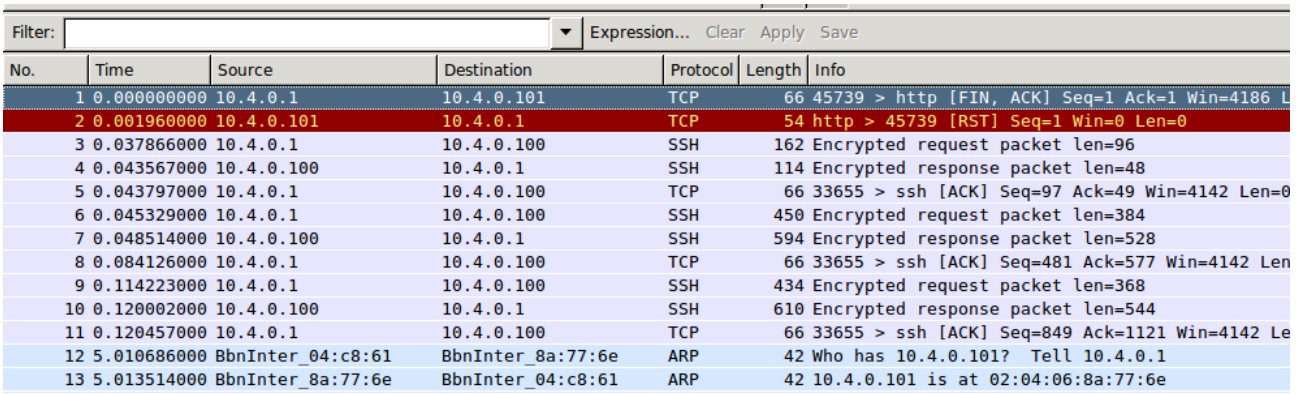
I 7.7 Démarrez une capture avec wireshark sur spy.



I 7.8 Dans epiphany, rentrez maintenant l'URL http://localhost:10000 puis validez. (Avec cette URL, epiphany se connecte au port 10000 de localhost (l'hôte admin lui-même).)



I 7.9 Arrêtez la capture sur spy.



Q 7.1 Que constatez-vous dans les paquets capturés ?

Dans les paquets capturés, on constate qu'il y a des paquets SSH crypté de type REQUEST et RESPONSE. On n'a donc pas accès aux paquets HTML de leur échange.

Conclusion :

Ce TP nous a permis de découvrir et comprendre le protocole SSH sous plusieurs aspects. Dans un premier temps, en comparant telnet et SSH, nous avons pu constater concrètement le danger de transmettre des mots de passe en clair sur un réseau, et comprendre pourquoi telnet est aujourd'hui abandonné au profit de SSH. Nous avons ensuite exploré les différentes fonctionnalités offertes par SSH : la connexion distante, la copie sécurisée de fichiers avec scp, l'authentification par clé publique et la création de tunnels pour chiffrer des protocoles non sécurisés.

La principale difficulté rencontrée était la compréhension du mécanisme des clés : pourquoi la régénération d'une clé côté serveur bloque la connexion côté client, et comment y remédier avec ssh-keygen -R. Une fois le principe de la paire clé publique/clé privée assimilé, la mise en place de l'authentification sans mot de passe est devenue claire.

Dans un contexte professionnel, SSH est un outil incontournable pour tout administrateur réseau ou système. Il permet d'administrer des serveurs à distance de manière sécurisée, d'automatiser des tâches via des scripts sans intervention humaine grâce à l'authentification par clé, et de sécuriser des communications normalement non chiffrées grâce aux tunnels. La maîtrise de SSH est donc une compétence fondamentale dans les métiers des réseaux et télécommunications.

GROT Yanis GROUPE 3

TP3 : Configuration d’un serveur web

Table des matières :

- Introduction
- Exercice 1 : Création d’un réseau
- Exercice 2 : Création du site www.p13.edu
- Exercice 3 : Création du site www.iutv.p13.edu
- Exercice 4 : Fichiers d’erreur et journaux
- Exercice 5 : Activation des scripts CGI
- Exercice 6 : Authentification

Introduction :

Ce travail pratique s'inscrit dans le cadre de l'étude des services réseaux, plus précisément l'administration d'un serveur web sous Linux en utilisant la technologie Apache2. L'objectif principal de ce TP est de comprendre et de mettre en œuvre le mécanisme des hôtes virtuels (Virtual Hosts), qui permet de faire cohabiter plusieurs sites web distincts sur une seule infrastructure physique. Pour atteindre cet objectif, nous utilisons l'outil de simulation réseau Marionnet, qui nous permet de configurer un serveur Apache et un poste client dans un environnement virtuel contrôlé. Le TP est structuré en plusieurs étapes progressives : nous commençons par la création du réseau et l'attribution des adresses IP, suivies par la configuration technique des sites www.p13.edu et www.iutv.p13.edu. La démarche inclut également la gestion avancée du serveur via la personnalisation des pages d'erreur et des journaux, l'activation de scripts dynamiques CGI pour le monitoring du système, et enfin, la mise en place d'une sécurité d'accès par authentification pour protéger les zones d'administration.

Exercice 1 : Création du réseau

Objectif : L’objectif est de mettre en place un réseau simple entre un client et un serveur web afin de préparer les manipulations Apache du TP.

Démarche

Étapes réalisées :

- Création d’un réseau Marionnet avec deux machines : client et www.

- Connexion des deux machines avec un câble croisé.

- Démarrage des équipements et ouverture des sessions root.

- Attribution manuelle des adresses IP.

- Vérification de la connectivité réseau.

Commandes utilisées

Configuration IP : ip addr, ip link set, ping

Exercice 2 : Création du site www.p13.edu

Objectif : L'objectif est de créer un premier site web hébergé par Apache et de configurer un hôte virtuel associé au nom www.p13.edu.

Démarche :

Étapes réalisées

- **Création du répertoire du site web.**
- **Création de la page HTML principale.**
- **Création du fichier de configuration Apache.**
- **Activation de l'hôte virtuel.**
- **Redémarrage du serveur Apache.**
- **Ajout du nom du site dans /etc/hosts sur le client.**
- **Vérification du fonctionnement avec un navigateur.**

Commandes utilisées : mkdir, nano, ln -s, restart

I 2.1 Créez le répertoire /var/www/p13

```
[0 root@serveur ~]$ mkdir /var/www/p13
[0 root@serveur ~]$ nano /var/www/p13/index.html
[0 root@serveur ~]$
```

I 2.2 Créez puis éditez un fichier HTML /var/www/p13/index.html avec le rendu de la figure ci-dessus.

```
<!DOCTYPE html>

<head>
        <meta charset="UTF-8">
        <title>Université Paris 13</title>
</head>
<body>
        <h1>Université Paris &"/h&>
        <p> Bienvenue sur le site web de l'Université Paris 13.</p>
        <p><a href="http://www.iutv.p13.edu">Aller sur le site web de l'IUT de Villetaneuse</a> </p>
</body>
</html>
```

I 2.3 Crééz puis éditez le fichier /etc/apache2/sites-available/p13.conf pour qu'il ait le contenu ci-dessus.

```
<VirtualHost *:80>
ServerName www.p13.edu
DocumentRoot /var/www/p13
DirectoryIndex index.html
</VirtualHost>
```

I 2.4 Créez le lien symbolique /etc/apache2/sites-enabled/p13.conf pointant vers le fichier créé en I 2.3.

```
nano /etc/apache2/sites-available/p13.conf
nano /etc/apache2/sites-available/p13.conf
ln -s /etc/apache2/sites-available/p13.conf /etc/apache2/sites-enabled/p13.conf
```

I 2.5 Redémarrez le serveur apache :

```
[0 root@serveur ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualified domain name, using 127.
0.0.1 for ServerName
apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
[0 root@serveur ~]$
```

I 2.6 Sur le client : éditez le fichier /etc/hosts pour ajouter une ligne associant l'IP 10.N.0.100 au nom www.p13.edu.

```
127.0.0.1       localhost
::1             localhost ip6-localhost ip6-loopback
fe00::0         ip6-localnet
ff00::0         ip6-mcastprefix
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters

127.0.0.1 client
10.4.0.100 www.p13.edu
```

I 2.7 Sur le client : ouvrez epiphany et entrez l'URL http://www.p13.edu pour vérifier que la page HTML s'affiche.

```
                                                    Universite Paris 13
                          Universite Paris 13

 Bienvenue sur le site web de l'Universite Paris 13

 Aller sur le site web de l'IUT de Villetaneuse
```

Exercice 3 — Création du site www.iutv.p13.edu

Objectif : L'objectif est d'héberger un second site web sur le même serveur Apache grâce aux hôtes virtuels.

Démarche :

Étapes réalisées :
- Création du répertoire du second site.
- Création d'une nouvelle page HTML.
- Création d'un second hôte virtuel Apache.
- Activation du site.
- Test du site avec curl.

Commandes utilisées : mkdir, nano, ln -s, curl

I 3.1 En suivant les étapes de l'exercice précédent, faites en sorte que le site www.iutv.p13.edu soit également servi par le serveur web. Ce site a les caractéristiques suivantes :
— fichier de conf. : /etc/apache2/sites-available/iutv.conf
— répertoire contenant ses fichiers : /var/www/iutv
— Le rendu de sa page d'accueil est donné sur la figure de droite. Le lien pointe sur l'URL http://www.p13.edu.

```
<!DOCTYPE html>

<head>
   <meta charset="UTF-8">
   <title>IUT de Villetaneuse</title>
</head>
<body>
   <h1>IUT de Villetaneuse</h1>
   <p>Bienvenue sur le site web de l'IUT de Villetaneuse.</p>
   <p><a href="http://www.p13.edu">Aller sur le site web de l'université</a></p>
</body>
</html>
```

```
<VirtualHost *:80>
  ServerName www.iutv.p13.edu
  DocumentRoot /var/www/iutv
  DirectoryIndex index.html
</VirtualHost>
```

```
[0 root@serveur ~]$ ln -s /etc/apache2/sites-available/iutv.conf /etc/apache2/sites-enabl
ed/iutv.conf
[0 root@serveur ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's f
ully qualified domain name, using 127.0.0.1 for ServerName
 ... waiting apache2: Could not reliably determine the server's fully qualified domain na
me, using 127.0.0.1 for ServerName
. ok
[0 root@serveur ~]$
```

```
127.0.0.1       localhost
::1             localhost ip6-localhost ip6-loopback
fe00::0         ip6-localnet
ff00::0         ip6-mcastprefix
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters

127.0.0.1 client
10.4.0.100 www.p13.edu
10.4.0.100 www.iutv.p13.edu
```



I 3.2 Depuis le client, récupérez, avec curl, le fichier à la racine du site www.iutv.p13.edu :

```
[0 root@client ~]$ curl -v www.iutv.p13.edu
* About to connect() to www.iutv.p13.edu port 80 (#0)
*   Trying 10.4.0.100...
* connected
* Connected to www.iutv.p13.edu (10.4.0.100) port 80 (#0)
> GET / HTTP/1.1
> User-Agent: curl/7.26.0
> Host: www.iutv.p13.edu
> Accept: */*
>
* additional stuff not fine transfer.c:1037: 0 0
* HTTP 1.1 or later with persistent connection, pipelining supported
< HTTP/1.1 200 OK
< Date: Mon, 04 May 2026 07:51:08 GMT
< Server: Apache/2.2.22 (Debian)
< Last-Modified: Mon, 04 May 2026 07:46:14 GMT
< ETag: "1a76a-127-650f91ffcbd80"
< Accept-Ranges: bytes
< Content-Length: 295
< Vary: Accept-Encoding
< Content-Type: text/html
< X-Pad: avoid browser bug
<
<!DOCTYPE html>

<head>
   <meta charset="UTF-8">
   <title>IUT de Villetaneuse</title>
</head>
<body>
   <h1>IUT de Villetaneuse</h1>
   <p>Bienvenue sur le site web de l'IUT de Villetaneuse.</p>
   <p><a href="http://www.p13.edu">Aller sur le site web de l'université</a></p>
</body>
</html>
* Connection #0 to host www.iutv.p13.edu left intact
* Closing connection #0
[0 root@client ~]$
```

Q 3.1 Quelle est la version d'HTTP utilisée par curl ?

La version d'HTTP utilisée par curl est : HTTP/1.1

Q 3.2 Quelle est le code de retour dans la réponse ?

Le code de retour dans la réponse est : 200 OK

Q 3.3 Donnez les noms des champs HTTP contenant les informations suivantes :

- Dans la requête : Host
- Dans la réponse (date de dernière modification) : Last-Modified
- Dans la réponse (format des données) : Content-Type

Exercice 4 — Fichiers d’erreur et journaux

Objectif : L’objectif est de personnaliser les pages d’erreur HTTP et de configurer les journaux Apache pour enregistrer les accès et erreurs.

Démarche

Étapes réalisées

- Création des pages d’erreur 403 et 404.

- Ajout des directives ErrorDocument.

- Création du répertoire de logs.

- Configuration des fichiers access.log et error.log.

- Génération volontaire d’erreurs HTTP.

Commandes utilisées : mkdir, nano, restart, cat

I 4.1 Créez le répertoire /var/www/iutv/erreurs.

I 4.2 Créez puis éditez dans ce répertoire un fichier 403.html avec le rendu de la figure ci-dessus.

```
<!DOCTYPE html>

<head>
   <meta xharset="UTF-8">
   <title>Erreur 403</title>
</head>
<body>
   <p>Erreur 403: accès non autorisé!!!</p>
   <p>Retour sur <a href="http://www.iutv.p13.edu">www.iutv.p13.edu</a></p>
</body>
</html>
```

I 4.3 Faites de même avec le fichier 404.html.

```
<!DOCTYPE html>

<head>
  <meta xharset="UTF-8">
  <title>Erreur 404</title>
</head>
<body>
  <p>Erreur 404: fichier introuvable!!!</p>
  <p>Retour sur <a href="http://www.iutv.p13.edu">www.iutv.p13.edu</a></p>
</body>
</html>
```

I 4.4 Dans /etc/apache2/sites-available/iutv.conf, ajoutez, dans la déclaration de l'hôte virtuel :

```
<VirtualHost *:80>
  ServerName www.iutv.p13.edu
  DocumentRoot /var/www/iutv
  DirectoryIndex index.html
</VirtualHost>
ErrorDocument 403 /erreurs/403.html
ErrorDocument 404 /erreurs/404.html
```

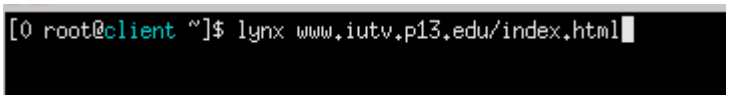
I 4.5 Redémarrez le serveur apache.

```
[0 root@serveur ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualifi
ed domain name, using 127.0.0.1 for ServerName
 ... waiting apache2: Could not reliably determine the server's fully qualified domain name, using 12
7.0.0.1 for ServerName
. ok
[0 root@serveur ~]$
```

4.6 Sur le client, faites les tests suivants dans epiphany :



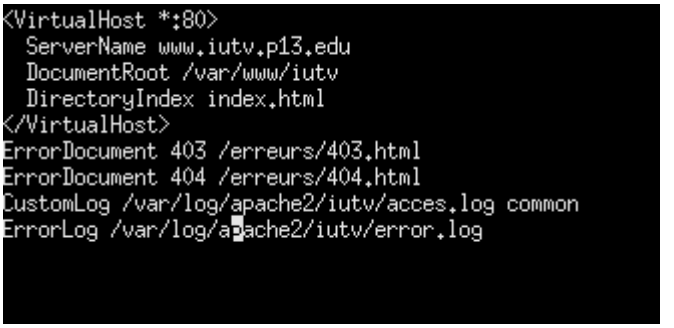
2 :



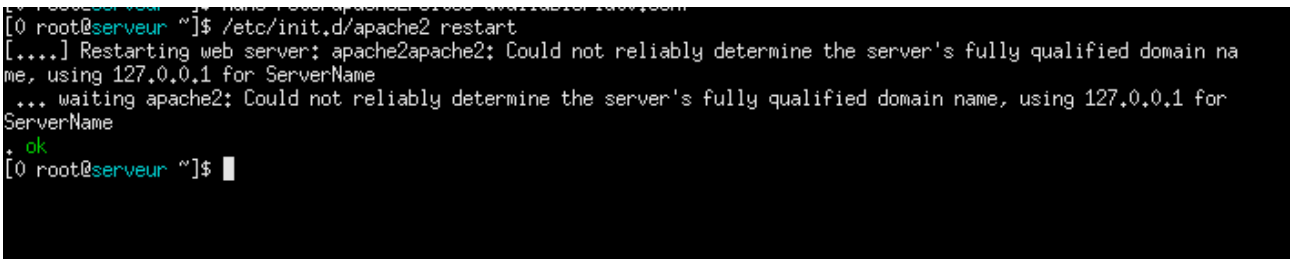


I 4.7 Créez le répertoire /var/log/apache2/iutv.

I 4.8 Dans /etc/apache2/sites-available/iutv.conf, ajoutez, dans la déclaration de l'hôte virtuel :



I 4.9 Redémarrez le serveur apache.



I 4.10 Sur le client, faites les tests suivants dans epiphany :





I 4.11 Affichez les contenus des deux fichiers (access.log et error.log) pour vérifier que vous avez bien de nouvelles lignes suite aux deux requêtes envoyées par le client à la question précédente.

```
[1 root@serveur ~]$ cat /var/log/apache2/access.log common
10.4.0.1 - - [04/May/2026:10:36:39 +0200] "GET / HTTP/1.0" 200 446 "-" "Lynx/2.8.8dev.12 libwww-FM/2.14 SSL-MM/1.4.1 GNUTL
S/2.12.18"
```

```
[1 root@serveur ~]$ cat /var/log/apache2/error.log
[Mon May 04 09:33:06 2026] [notice] Apache/2.2.22 (Debian) configured -- resuming normal operations
[Mon May 04 09:47:26 2026] [notice] caught SIGTERM, shutting down
[Mon May 04 09:47:28 2026] [notice] Apache/2.2.22 (Debian) configured -- resuming normal operations
[Mon May 04 10:17:20 2026] [notice] caught SIGTERM, shutting down
[Mon May 04 10:17:21 2026] [notice] Apache/2.2.22 (Debian) configured -- resuming normal operations
[Mon May 04 10:21:01 2026] [error] [client 10.4.0.1] File does not exist: /var/www/iutv/introuvable
[Mon May 04 10:24:22 2026] [error] [client 10.4.0.1] (13)Permission denied: file permissions deny server access: /var/www/iutv/ind
ex.html
[Mon May 04 10:25:12 2026] [error] [client 10.4.0.1] (13)Permission denied: file permissions deny server access: /var/www/iutv/ind
ex.html, referer: http://www.iutv.p13.edu/index.html
[Mon May 04 10:31:37 2026] [notice] caught SIGTERM, shutting down
```

Exercice 5 — Activation des scripts CGI

Objectif : L'objectif est d'autoriser l'exécution de scripts CGI sur Apache afin de générer du contenu dynamique.

Démarche

Étapes réalisées

- Création du répertoire cgi.

- Création d'un script shell CGI.

- Ajout des directives Apache pour autoriser les CGI.

- Activation du module CGI. Test du script via un navigateur.

Commandes utilisées : mkdir, nano, chmod, ln -s, restart

I 5.1 Créez les répertoires /var/www/iutv/admin et /var/www/iutv/admin/cgi.

```
[0 root@serveur ~]$ mkdir /var/www/iutv/admin
[0 root@serveur ~]$ mkdir /var/www/iutv/admin/cgi
[0 root@serveur ~]$
```

I 5.2 Créez dans ce dernier le fichier test-services.sh et rendez le exécutable.

```
[0 root@serveur ~]$ mkdir /var/www/iutv/admin
[0 root@serveur ~]$ mkdir /var/www/iutv/admin/cgi
[0 root@serveur ~]$ touch /var/www/iutv/admin/cgi/test-services.sh
[0 root@serveur ~]$ chmod +x /var/www/iutv/admin/cgi/test-services.sh
[0 root@serveur ~]$
```

I 5.3 Éditez ce fichier pour que ce soit un script CGI retournant du texte brut et affichant l'état des services apache2,cron, rsyslog et ssh. Utilisez de préférence une boucle plutôt que d'écrire 4 fois l'instruction /etc/init.d/...

```
#!/bin/bash
echo "Content-Type: text/plain"
echo ""

for service in apache2 cron rsyslog ssh
do
    /etc/init.d/$service status
done
```

I 5.4 Testez votre script dans le terminal.

I 5.5 Dans /etc/apache2/sites-available/iutv.conf, ajoutez, dans la déclaration de l'hôte virtuel :

```
<VirtualHost *:80>
  ServerName www.iutv.p13.edu
  DocumentRoot /var/www/iutv
  DirectoryIndex index.html
</VirtualHost>
ErrorDocument 403 /erreurs/403.html
ErrorDocument 404 /erreurs/404.html
CustomLog /var/log/apache2/iutv/acces.log common
ErrorLog /var/log/apache2/iutv/error.log

<Directory /var/www/iutv/admin/cgi>
   Options +ExecCGI
   AddHandler cgi-script  .sh
</Directory>
```

I 5.6 Créez le lien symbolique /etc/apache2/mods-enabled/cgi.load pointant vers le fichier /etc/apache2/mods-available/cgi.load.

```
[0 root@serveur ~]$ ln -s /etc/apache2/mods-available/cgi.load /etc/apache2/mods-enabled/cgi.load
[0 root@serveur ~]$
```

I 5.7 Redémarrez le serveur apache.

```
[0 root@serveur ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for Server
Name
 ... waiting apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
[0 root@serveur ~]$
```

I 5.8 Sur client, testez en entrant l'URL http://www.iutv.p13.edu/admin/cgi/test-services.sh dans epiphany.

```
Apache2 is running (pid 2200).
cron is running.
rsyslogd is not running ... failed!
sshd is not running ... failed!
```

Exercice 6 — Authentification

Objectif : L'objectif est de sécuriser l'accès à un répertoire web grâce à une authentification Apache par mot de passe.

Démarche

Étapes réalisées

- Création des utilisateurs Apache avec htpasswd.

- Configuration des directives d'authentification.

- Activation du module auth_basic.

- Test de connexion avec différents utilisateurs.

Commandes utilisées : htpasswd, ln -s, restart

I 6.1 Créez un mot de passe pour admin :

```
[0 root@serveur ~]$ htpasswd -c /etc/apache2/passwd admin
New password:
Re-type new password:
Adding password for user admin
[0 root@serveur ~]$ htpasswd /etc/apache2/passwd anna
New password:
Re-type new password:
Adding password for user anna
[0 root@serveur ~]$
```

Dans /etc/apache2/sites-available/iutv.conf, ajoutez, dans la déclaration de l'hôte virtuel :

```
<Directory /var/www/iutv/admin>
   AuthType Basic
   AuthUserFile /etc/apache2/passwd
   AuthName "Fichiers d'admin"
   Require user admin
</Directory>
```

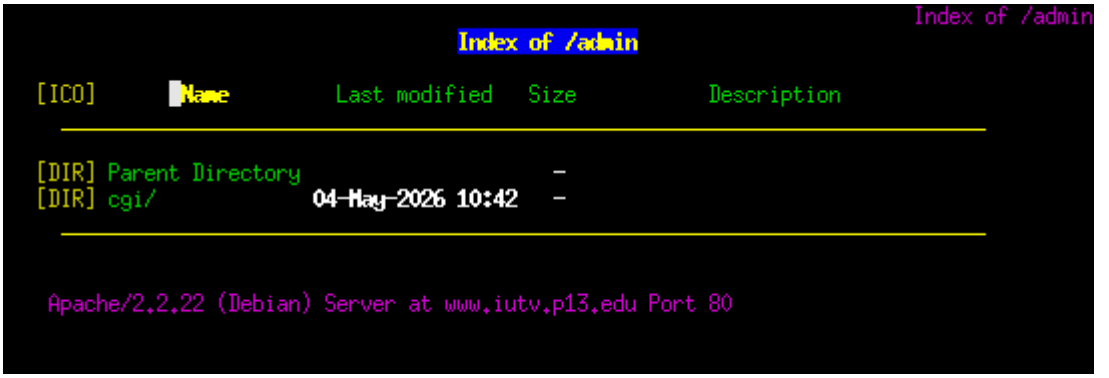
I 6.4 Créez le lien symbolique /etc/apache2/mods-enabled/auth_basic.load pointant vers le fichier /etc/apache2/mods-available/auth_basic.load. (Il se peut que le lien existe déjà auquel cas vous pourrez ignorer le message d'erreur.)

```
[0 root@serveur ~]$ ln -s /etc/apache2/mods-available/auth_basic.load /etc/apache2/mods-enabled/
ln: failed to create symbolic link `/etc/apache2/mods-enabled/auth_basic.load': File exists
[1 root@serveur ~]$
```

I 6.5 Redémarrez le serveur apache.

```
[1 root@serveur ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
 ... waiting apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
[0 root@serveur ~]$
```

I 6.6 Sur le client, testez en entrant l'URL http://www.iutv.p13.edu/admin dans epiphany que





Conclusion :

En conclusion, ce TP a permis de maîtriser les fondamentaux de l'administration d'un serveur web Apache à travers la mise en place d'une architecture multi-sites reposant sur les hôtes virtuels, tout en synthétisant des apprentissages cruciaux sur la gestion des répertoires, la configuration des fichiers système et l'interaction entre client et serveur. Bien que la progression ait été marquée par des difficultés techniques, notamment lors de la configuration des droits d'accès pour les scripts CGI et la mise en place des liens symboliques entre les répertoires "available" et "enabled", ces obstacles ont été surmontés grâce à une analyse rigoureuse des journaux d'erreurs (error logs) et au redémarrage systématique des services pour valider chaque modification. Ces compétences sont directement transposables dans un contexte professionnel, car la capacité à déployer des serveurs web sécurisés, à automatiser des tâches d'administration via des scripts et à restreindre l'accès à des zones sensibles par authentification constitue le socle du métier d'administrateur système et réseau, garantissant ainsi la disponibilité et la fiabilité des services numériques au sein d'une infrastructure d'entreprise.

GROT Yanis GROUPE 3

TP4 : Configuration d’un serveur FTP répliqué

Table des matières :

- 1) Introduction
- 2) Exercice 1 Création du réseau et configuration des interfaces
- 3) Exercice 2 Test et configuration du serveur FTP
- 4) Exercice 3 : Configuration du proxy
- 5) Exercice 4 : Prise en main de l’outil rsync
- 6) Exercice 5 : Configuration et test du serveur miroir
- 7) Exercice 6 : Mise à jour automatique du serveur miroir

Introduction :

Ce travail pratique porte sur l'étude et la mise en œuvre d'une architecture de stockage redondante à travers la configuration d'un serveur FTP répliqué et l'utilisation d'un serveur miroir. L'objectif principal est d'apprendre à assurer la haute disponibilité et la synchronisation des données entre deux serveurs (un serveur FTP principal et un serveur miroir) tout en facilitant l'accès pour l'utilisateur final via un proxy inverse (reverse proxy). Pour ce faire, nous utilisons l'environnement de simulation Marionnet afin de manipuler des outils professionnels tels que le serveur FTP ProFTPD, le module de répartition de charge d'Apache (mod_proxy_balancer) et l'utilitaire de synchronisation rsync. Le TP se déroule en plusieurs phases clés : il débute par la mise en place de l'infrastructure réseau et la configuration des interfaces, suivies de la validation du service FTP et de l'installation d'un proxy pour masquer la complexité du réseau au client. Ensuite, nous expérimentons la réplication manuelle avec rsync avant de configurer le serveur miroir et de mettre en place un mécanisme de mise à jour automatique via des scripts Shell et des tâches planifiées Cron, garantissant ainsi une synchronisation transparente et efficace des fichiers.

Exercice 1 : Création du réseau et configuration des interfaces

Objectif : L'objectif est de créer l'infrastructure réseau du TP et de configurer manuellement les adresses IP des différentes machines.

Démarche :

Étapes réalisées

- Téléchargement et ouverture du projet Marionnet contenant les serveurs FTP.

- Ajout des machines client et www.

- Démarrage des équipements et ouverture des sessions root.

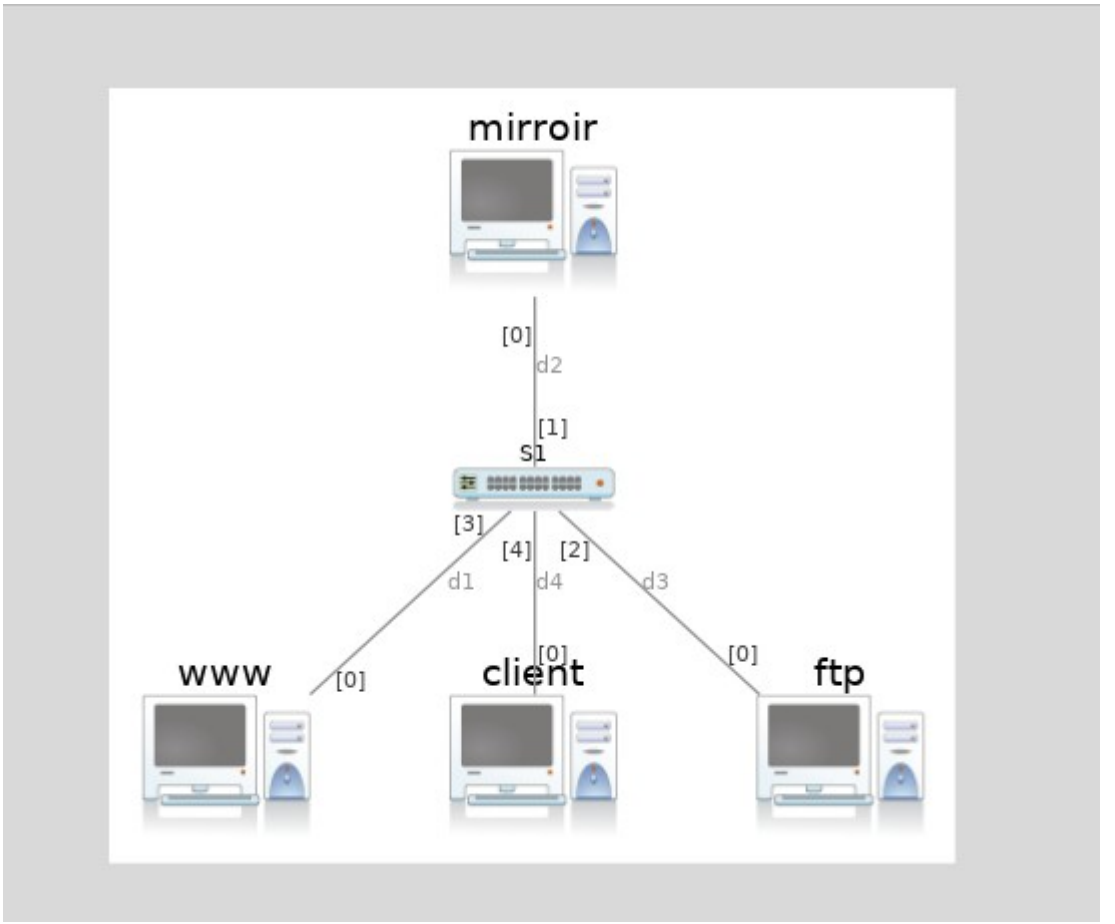
- Attribution des adresses IP statiques.

- Vérification de la connectivité réseau avec ping.

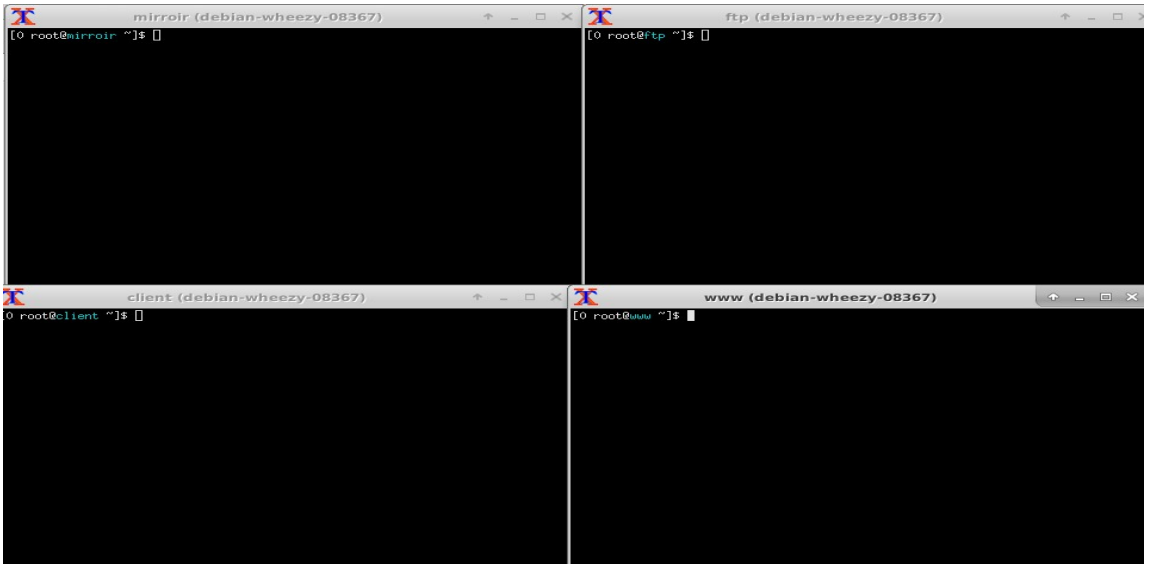
Commandes utilisées : ip addr, ping, ip link set

I 1.2 Lancez marionnet et ouvrez le projet téléchargé

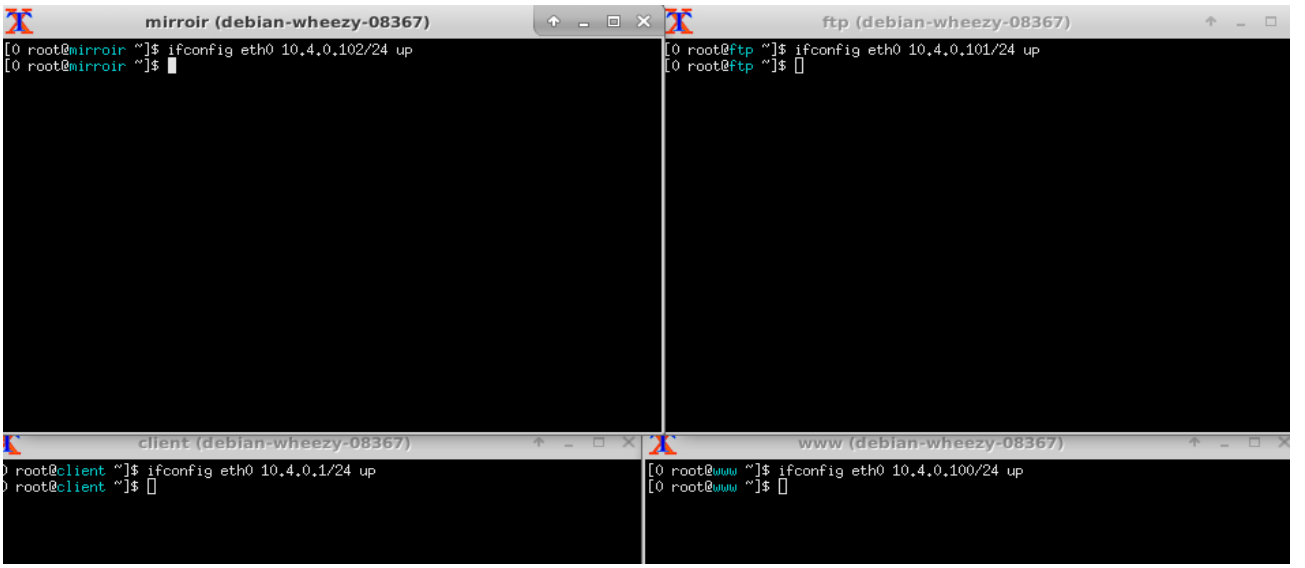
I 1.3 Ajoutez les équipements pour obtenir le réseau de la figure en introduction.



I 1.4 Démarrez tous les équipements et ouvrez une session root (mot de passe = root) sur chaque hôte.

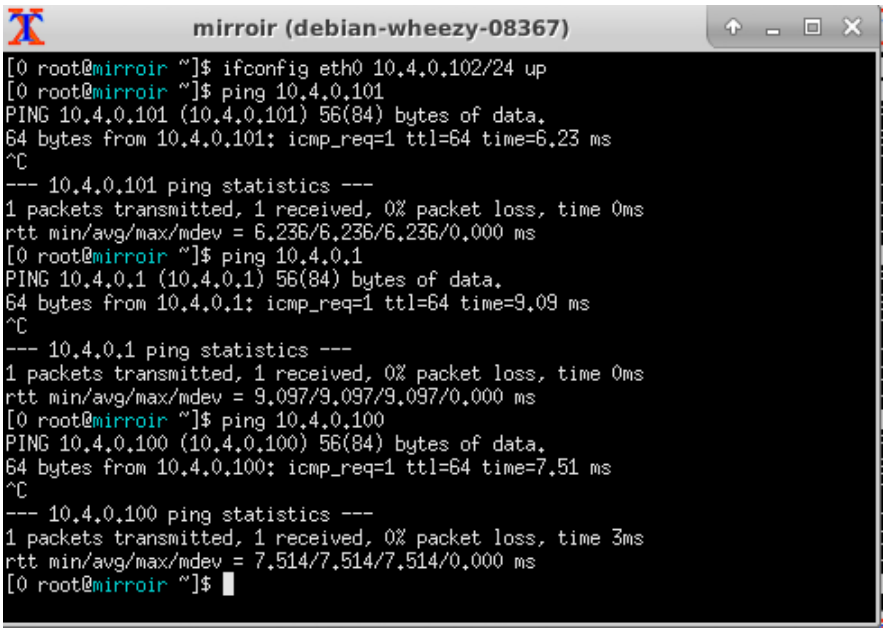


1.5 À l'aide de la commande ip (voir exercice 2 du TP 1), attribuez les IP aux interfaces.

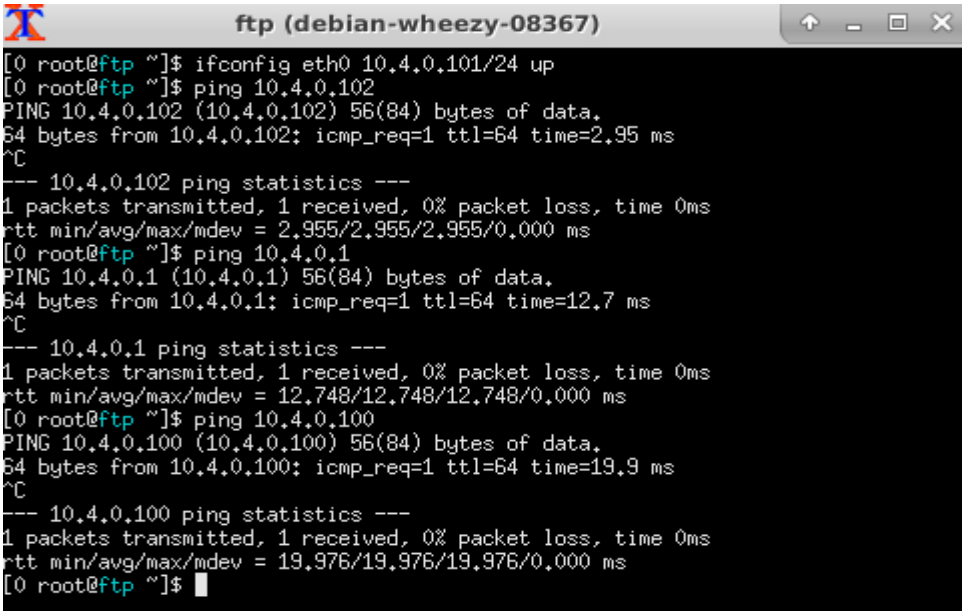


1.6 Vérifiez que les hôtes peuvent bien s'échanger des messages ping.

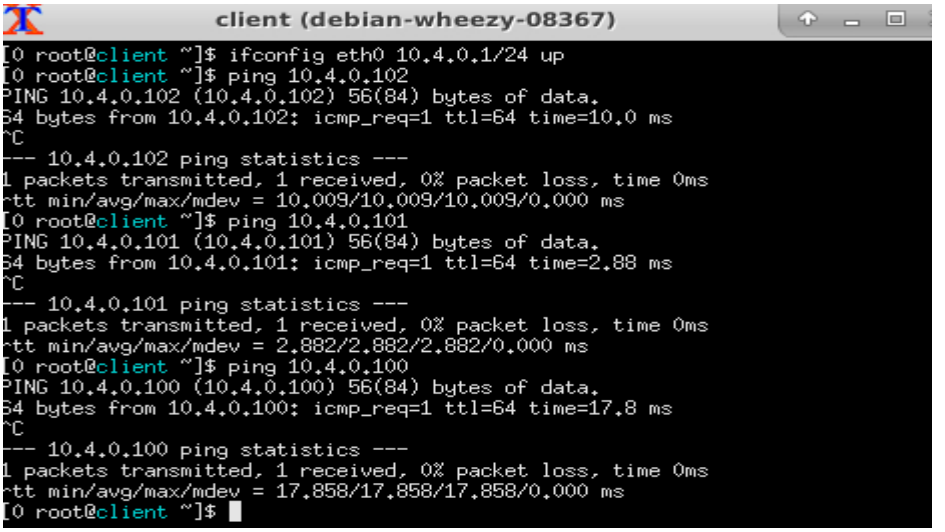
Pour mirroir :



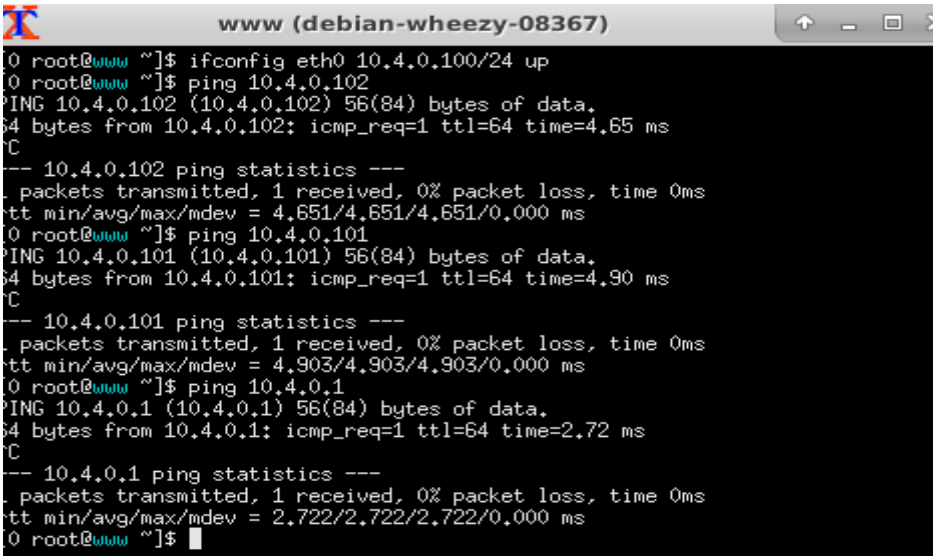
Pour ftp :



Pour client :



Pour www :



Exercice 2 — Test et configuration du serveur FTP

Objectif : L'objectif est de découvrir le fonctionnement du protocole FTP et de configurer l'accès anonyme au serveur FTP.

Démarche :

Étapes réalisées :

- Création de fichiers sur le serveur FTP et sur le client.

- Connexion FTP avec le compte student.

- Test des commandes FTP (get, put, mkdir, cd).

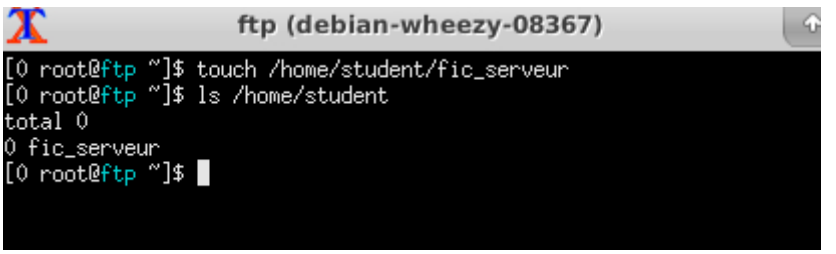
- Activation du mode anonymous.

-Création de fichiers accessibles anonymement.

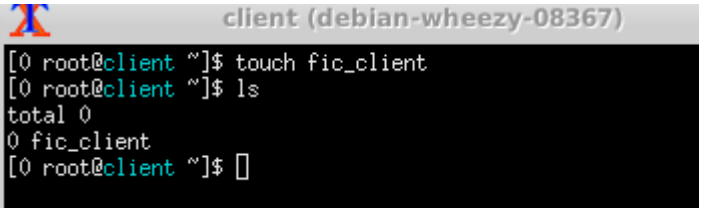
-Test de téléchargement avec un utilisateur anonyme.

Commandes utilisées : ftp, get, mkdir, cd, put, exit, nano, restart, mkdir, touch

2.1 Sur ftp, créez le fichier /home/student/fic_serveur.



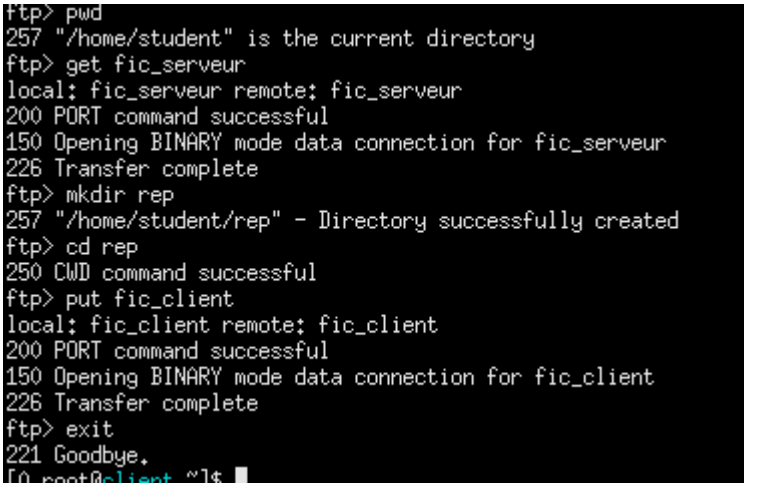
I 2.2 Sur le client, créez le fichier fic_client dans le répertoire courant.



2.3 Sur client, connectez vous en ftp sur l'hôte ftp :



I 2.4 Testez et commentez les commandes suivantes dans la session ftp du client :



Q 2.1 Dans quel répertoire se trouvent les fichiers sur le client ?

Dans le répertoire courant.

Q 2.2 Dans quels répertoires se trouvent les fichiers sur ftp ?

Dans *home*/student/rep/

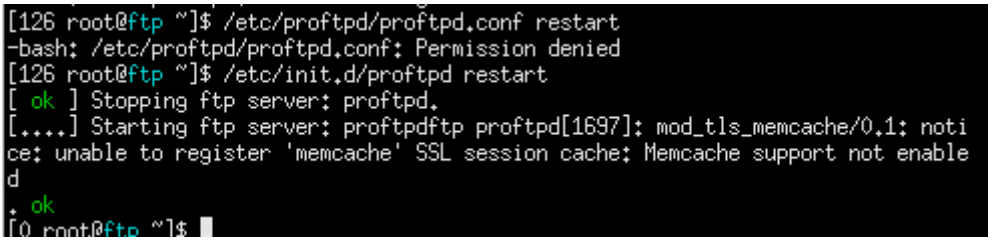
I 2.5 Éditez le fichier de configuration du service proftpd (/etc/proftpd/proftpd.conf).

2.6 Trouvez et décommentez (supprimez le caractère dièse en début de ligne) dans le fichier la section permettant l'accès anonymous (<Anonymous ~ftp> ... </Anonymous>).





I 2.7 Redémarrez le service proftpd.



I 2.8 Créez le répertoire /srv/ftp/rt



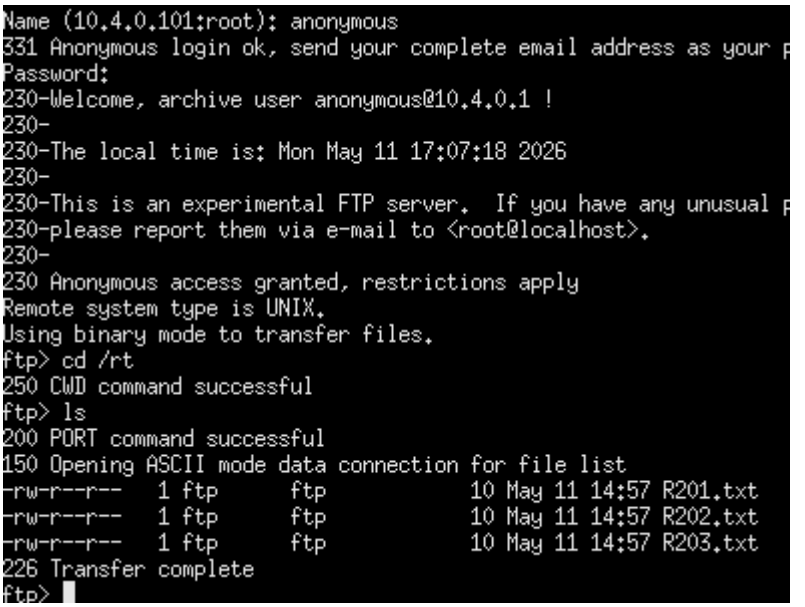
I 2.9 Créez, dans ce nouveau répertoire, les fichiers R201.txt, R202.txt et R203.txt.



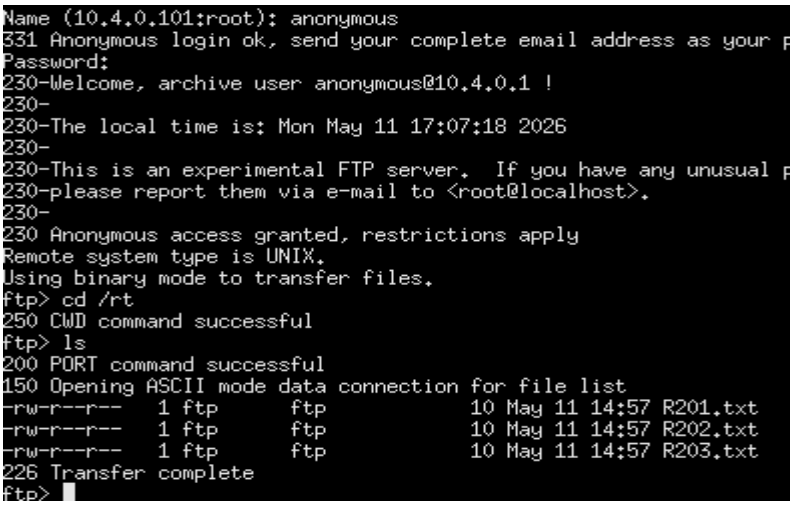
I 2.10 Éditez ces fichiers pour y ajouter du contenu (peu importe lequel).

Nous avons écrit dans les fichiers : test R201, R202, R203 respectivement.

I 2.11 Connectez vous en ftp sur l'hôte ftp. Entrez le nom anonymous et un mot de passe quelconque. (Aucun contrôle du mot de passe n'est effectué pour anonymous.)



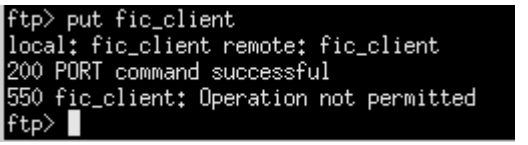
I 2.12 Déplacez vous dans le répertoire rt du serveur.



2.13 Téléchargez le fichier R203.txt.



I 2.14 Essayez de déposer le fichier fic_client du client dans le répertoire courant du serveur.



Q 2.3 Est-ce que la dernière commande a fonctionné ? Pourquoi ?

Non, elle n’a pas fonctionné car anonymous n’a pas les permissions nécessaires pour déposer le fichier.

I 2.15 Fermez la session ftp (exit).



Exercice 3 — Configuration du proxy

Objectif : L'objectif est de configurer Apache comme proxy FTP afin de permettre aux clients d'accéder au serveur FTP via HTTP.

Démarche

Étapes réalisées :

- Activation des modules Apache proxy et proxy_ftp.

- Création d'un hôte virtuel Apache.

- Configuration de ProxyPass.

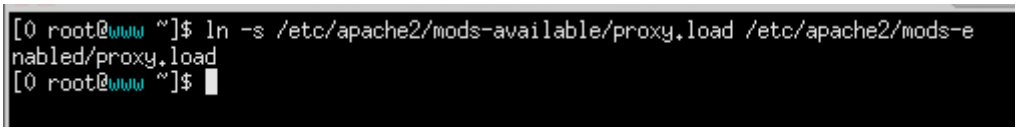
- Activation du site.

-Ajout du nom DNS local sur le client.

-Test du proxy dans le navigateur.

Commandes utilisées : ln -s, nano, restart,

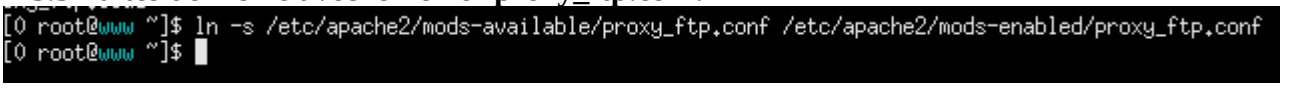
I 3.1 Créez le lien symbolique /etc/apache2/mods-enabled/proxy.load pointant vers le fichier /etc/apache2/mods-available/proxy.load.



I 3.2 Faites de même avec le fichier proxy_ftp.load.



I 3.3 Faites de même avec le fichier proxy_ftp.conf.



3.4 Créez le fichier /etc/apache2/sites-available/ftp.

I 3.5 Éditez ce fichier pour qu'il ait le contenu suivant :

```
<VirtualHost *:80>
  ServerName ftp.iutv.edu
  ProxyPass / ftp://10.4.0.101/
</VirtualHost>
```

3.6 Créez le lien symbolique /etc/apache2/sites-enabled/ftp pointant vers le fichier /etc/apache2/sites-available/ftp.

```
[0 root@www ~]$ ln -s /etc/apache2/sites-available/ftp /etc/apache2/sites-enabled/ftp
[0 root@www ~]$
```

I 3.7 Redémarrez le serveur apache2.

```
[0 root@www ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualified doma
in name, using 127.0.0.1 for ServerName
apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerNa
me
. ok
[0 root@www ~]$
```

I 3.8 Rajoutez dans /etc/hosts une ligne associant l'IP 10.N.0.100 et le nom ftp.iutv.edu.

```
127.0.0.1 client
10.4.0.100 ftp.iutv.edu
```

3.9 Lancez le navigateur web epiphany et entrez l'URL http://ftp.iutv.edu.

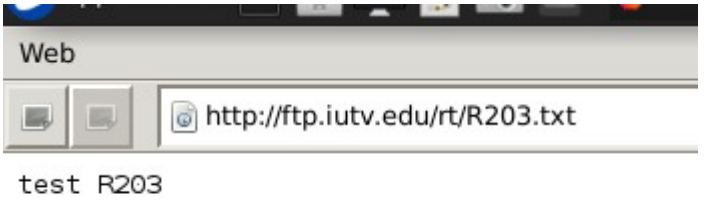
Directory of http://ftp.iutv.edu/

Welcome, archive user anonymous@10.4.0.100 !

```
drwxr-xr-x   3 ftp      ftp          4096 May 11 14:54 .
drwxr-xr-x   3 ftp      ftp          4096 May 11 14:54 ..
drwxr-xr-x   2 ftp      ftp          4096 May 11 14:56 rt
-rw-r--r--   1 ftp      ftp           170 May 19  2015 welcome.msg
```

Apache/2.2.22 (Debian) Server at ftp.iutv.edu Port 80

I 3.10 Vérifiez que vous pouvez bien aller dans le répertoire rt et télécharger le fichier R203.txt.



Exercice 4 — Prise en main de l'outil rsync

Objectif : L'objectif est d'apprendre à synchroniser automatiquement des fichiers entre deux serveurs grâce à rsync.

Démarche :

Étapes réalisées

- Démarrage du service SSH sur le serveur FTP.

- Première synchronisation du répertoire /srv/ftp.

- Observation du fonctionnement incrémental de rsync.

-Modification et création de fichiers.

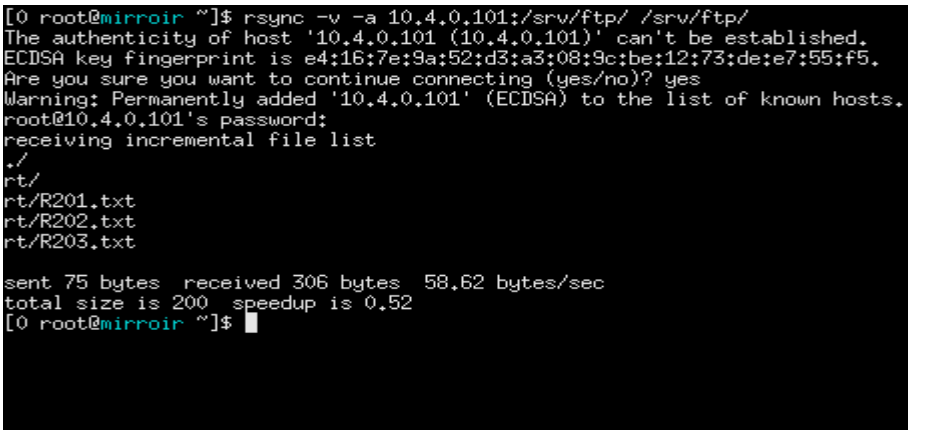
- Nouvelle synchronisation.

Commandes utilisées : restart, rsync, touch

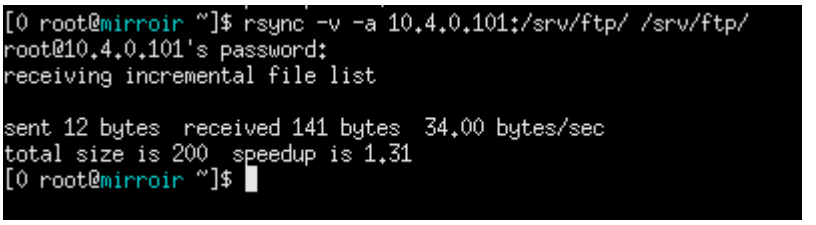
I 4.1 Sur ftp, démarrez le service ssh



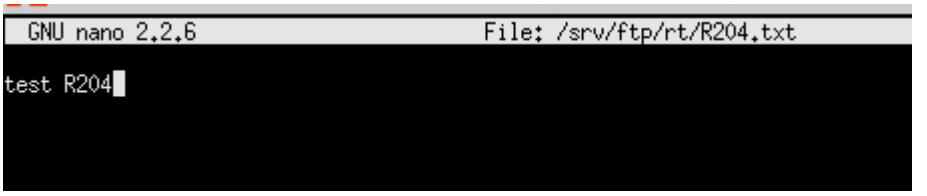
4.2 Sur miroir, lancez une synchronisation du répertoire /srv/ftp avec celui de l'hôte ftp (sans oublier les / à la fin des chemins) :



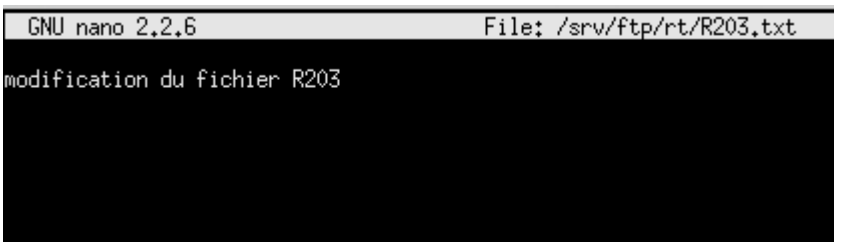
I 4.3 Relancez la commande de l'instruction I 4.2



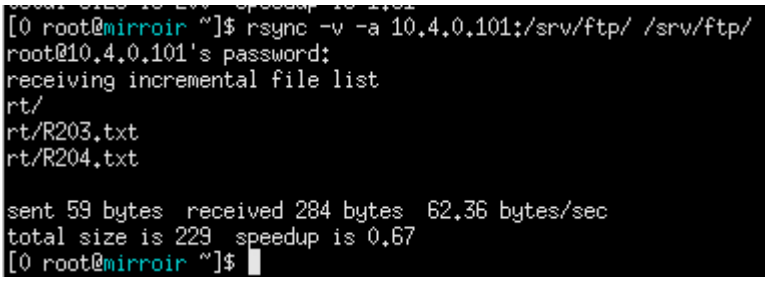
I 4.4 Sur ftp, créez le fichier /srv/ftp/rt/R204.txt



I 4.5 Sur ftp, modifiez le contenu du fichier /srv/ftp/rt/R203.txt.



I 4.6 Relancez la commande de l'instruction I 4.2



Exercice 5 — Configuration et test du serveur miroir

Objectif : L'objectif est de mettre en place un serveur miroir capable de remplacer automatiquement le serveur FTP principal en cas de panne.

Démarche

Étapes réalisées

-Copie de la configuration proftpd.

-Redémarrage du service FTP sur le miroir.

- Activation des modules Apache proxy_balancer.

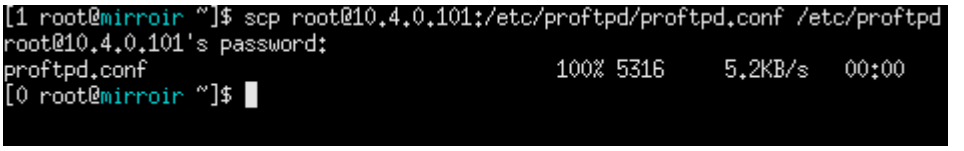
- Modification de la configuration Apache pour intégrer les deux serveurs FTP.

- Simulation d'une panne du serveur principal.

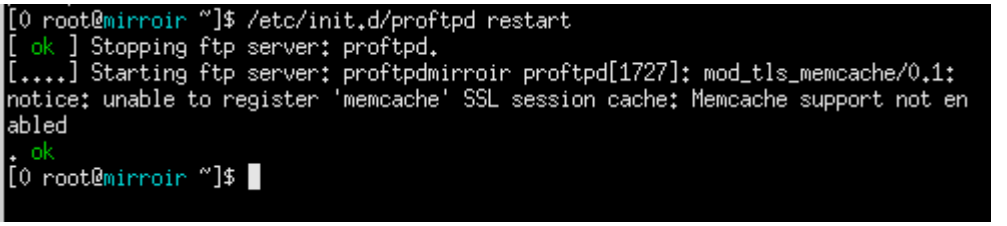
- Vérification du basculement automatique vers le miroir.

Commandes utilisées : scp, ln -s, nano, stop

I 5.1 Copiez, avec scp, le fichier /etc/proftpd/proftpd.conf du serveur ftp dans le répertoire /etc/proftpd.



I 5.2 Redémarrez le serveur proftpd



I 5.3 Créez le lien symbolique /etc/apache2/mods-enabled/proxy_balancer.load pointant vers le fichier /etc/apache2/mods-available/proxy_balancer.load.

```
[0 root@www ~]$ ln -s /etc/apache2/mods-available/proxy_balancer.load /etc/apache2/mods-enabled/proxy_balancer.loa
d
[0 root@www ~]$
```

I 5.4 Faites de même avec le fichier proxy_balancer.conf.

```
[0 root@www ~]$ ln -s /etc/apache2/mods-available/proxy_balancer.conf /etc/apache2/mods-enabled/proxy_balancer.conf
[0 root@www ~]$
```

I 5.5 Éditez le fichier de configuration d'apache :

— commentez la ligne ProxyPass ... qu'on avait écrite précédemment ;
— et ajoutez les lignes suivantes dans la définition de l'hôte virtuel :

```
<VirtualHost *:80>
  ServerName ftp.iutv.edu
#  ProxyPass / ftp://10.4.0.101/
</VirtualHost>

ProxyPass / balancer://serveurs/
 <Proxy balancer://serveurs/>
    BalancerMember ftp://10.4.0.101
    BalancerMember ftp://10.4.0.102 status=+H
</Proxy>
```

I 5.6 Redémarrez le serveur apache2

```
[0 root@www ~]$ /etc/init.d/apache2 restart
[....] Restarting web server: apache2apache2: Could not reliably determine the server's fully qualified domain name, using 1
27.0.0.1 for ServerName
 ... waiting apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
[0 root@www ~]$
```

I 5.7 Sur le client, lancez epiphany et entrez l'URL http://ftp.iutv.edu/rt/R203.txt.



modification du fichier R203

I 5.8 Sur ftp, arrêtez le service proftpd.



5.9 Sur le client, rechargez la page.



modification du fichier R203

Exercice 6 — Mise à jour automatique du serveur miroir

Objectif : L'objectif est d'automatiser la synchronisation du serveur miroir afin que les données restent toujours identiques à celles du serveur principal.

Démarche

Étapes réalisées

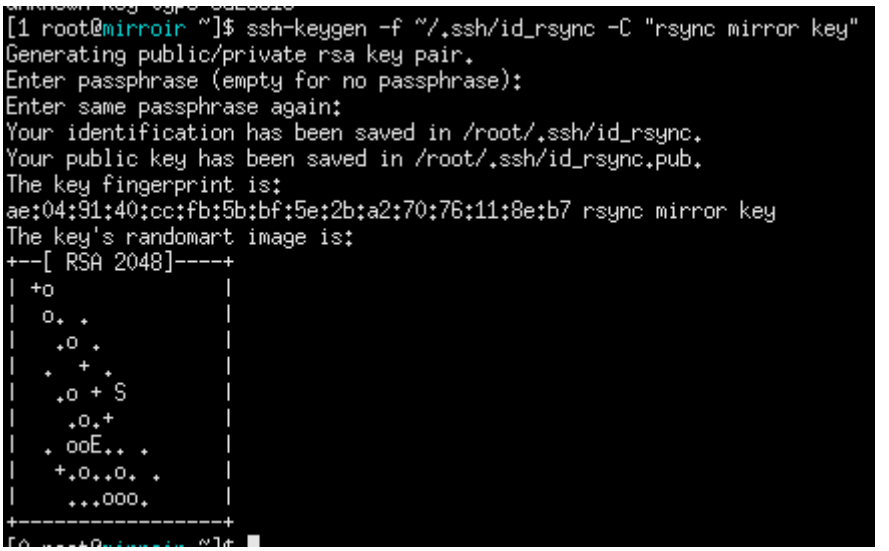
- Mise en place d'une authentification SSH par clé.

- Configuration d'une tâche automatique avec cron.

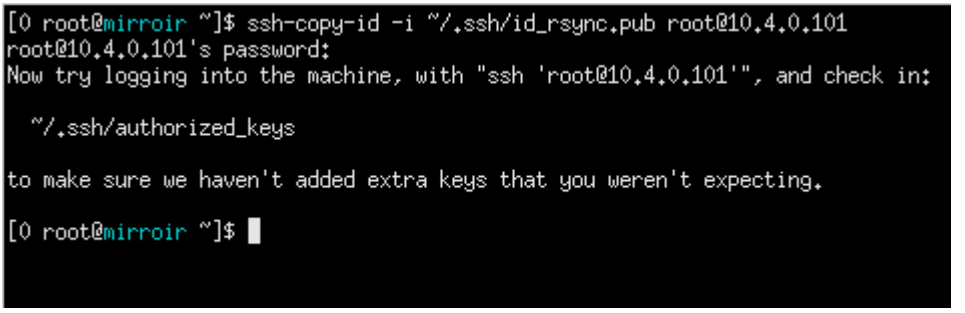
- Test de synchronisation automatique après modification des fichiers.

Commandes utilisées : ssh-keygen, scp, crontab

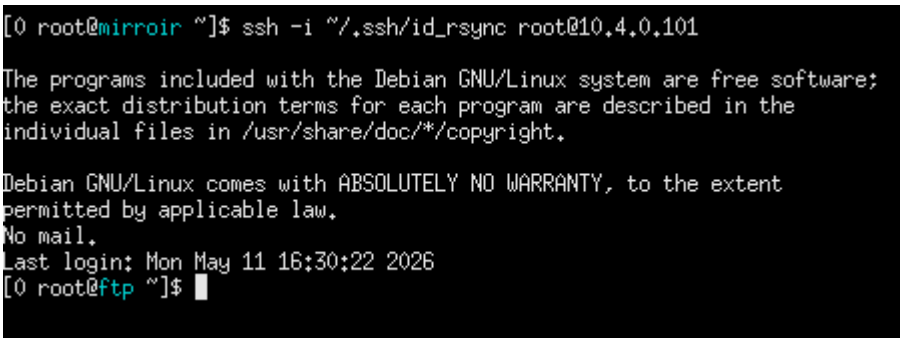
Générer une paire de clés SSH sur le serveur miroir :



Copier la clé publique sur le serveur FTP :



test de la connexion sans mot de passe :



Script :

```
#!/bin/bash

SRC="/srv/miroir/"
DEST="root@10.4.0.101:/srv/ftp/"

rsync -avz --delete -e "ssh -i /home/miroiruser/.ssh/id_rsync" "$SRC"  "$DEST"
```

Rendre le script executable :

```
[1 root@mirroir ~]$ chmod +x /usr/local/bin/rsync_mirroir.sh
[0 root@mirroir ~]$
```

Config crontab avec crontab -e :

```
0 * * * * /usr/local/bin/rsync_mirroir.sh >> /var/log/rsync_mirroir.log 2>&1
```

Conclusion :

En conclusion, ce TP a permis de maîtriser la mise en œuvre d'une architecture de stockage hautement disponible à travers la configuration d'un serveur FTP répliqué et l'utilisation d'un serveur miroir. Les apprentissages ont porté sur des concepts fondamentaux tels que le fonctionnement du protocole FTP avec ProFTPD, la mise en place d'un proxy inverse sous Apache pour masquer la structure interne du réseau, ainsi que la synchronisation de données critiques entre serveurs. Durant la réalisation, plusieurs difficultés techniques ont été rencontrées, notamment lors de la configuration du module de répartition de charge (balancer) et de l'établissement d'une connexion SSH sans mot de passe entre le serveur miroir et le serveur principal pour automatiser les transferts. Ces obstacles ont été surmontés en analysant rigoureusement les fichiers de configuration, en vérifiant la validité des liens symboliques dans les répertoires d'Apache et en testant manuellement les scripts de synchronisation rsync avant leur automatisation. Ce travail est particulièrement utile dans un contexte professionnel car la réplication de services et la sauvegarde automatisée via des tâches Cron sont des pratiques standards pour garantir la continuité de service et la protection des données en cas de panne matérielle, des compétences indispensables pour tout administrateur système garantissant la fiabilité d'une infrastructure informatique.